

A Little R Survival Kit

Eric S. Ho, Ph.D.

2026-06-16

Table of contents

Preface	5
Usage	5
RStudio	5
I Part I	7
1 Basic R	8
Learning Objectives:	8
1.1 R Variables	8
1.2 R Objects	9
1.2.1 Data Types	9
1.2.2 Atomic and Composite Variables	10
1.2.3 Vectors	12
1.2.4 Factors/Categorical Variables	13
1.2.5 Data Frames	14
1.3 Descriptive Statistics	19
1.3.1 Non-robust Statistics	19
1.3.2 Robust Statistics	20
1.3.3 summary() Function	22
1.4 Utility Functions	23
1.4.1 Inspection Functions	23
1.4.2 Printing and Formatting	24
1.4.3 seq() - create a series of numbers	25
1.4.4 with()	28
1.5 Summary	29
2 Data Wrangling	30
Learning Objectives:	30
2.1 Base R	30
2.1.1 Select Rows by Condition(s)	30
2.1.2 Random Sampling	34
2.1.3 Sort Items	35
2.1.4 Rearranging Columns	39
2.1.5 Focus on a Subset of Columns	39

2.1.6	Add a New Column	40
2.1.7	Remove a Column	41
2.1.8	Tabulation	41
2.2	tidyverse	42
2.2.1	Activate tidyverse	43
2.2.2	Select Rows by Condition(s)	44
2.2.3	Select Rows Randomly	45
2.2.4	Sort Rows in a Data Frame	46
2.2.5	Reorganize columns of a data frame	48
2.2.6	Focus on a Subset of Columns	48
2.2.7	Add a New Column	49
2.2.8	Remove a Column	49
2.2.9	Rename a Column	49
2.2.10	Pipe	50
2.2.11	Tabulation and Summarize	52
2.2.12	A Summary of tidyverse functions	54
2.3	Summary	55
3	Data Visualization	56
	Learning Objectives:	56
3.1	Types of Graphs	56
3.2	Base R Graphics	57
3.2.1	One Numeric Variable	57
3.2.2	One Categorical Variable	62
3.2.3	One Categorical and One Numeric Variables	67
3.2.4	Contingency Table	78
3.2.5	Two Numeric Variables	82
3.2.6	Multipanel Scatter Plot	85
3.3	ggplot2	86
3.3.1	Grammar of ggplot2	87
3.3.2	One Numeric Variable	87
3.3.3	One Categorical Variable	96
3.3.4	One Categorical and One Numeric Variables	102
3.3.5	Contingency Table	108
3.3.6	Two Numeric Variables	110
3.3.7	Multipanel Scatter Plot	111
3.4	Summary	112
II	Part II	114
4	Dive Deeper	115
	Learning Objectives:	115

4.1	Recreate a Summary Table in R	115
4.2	Create a small data frame with data in R	118
4.2.1	Use <code>data.frame()</code> Function	118
4.2.2	Use <code>read.table()</code> Function	119
4.3	Load datasets into R	120
4.3.1	<code>read.csv()</code> Function	121
4.3.2	Import data in RStudio	121
4.4	<code>group_by()</code>	121
4.5	Different <code>apply</code> Functions	121
4.6	Wide to Long	121

Preface

In the 21st century, nobody does statistics by pen and paper. Those statistical tables found at the back of statistics textbooks belong in the museum. Statistics you hear from the news and research are produced by computers. This book uses R. You may ask why not using a spreadsheet, such as Microsoft Excel or Google Sheets. While they are excellent tools - easy to use, they have significant shortcomings. In particular, they lack of reproducibility, as they operate through point-clicking, drag-and-drop, etc.

This book intends to provide readers with the bare minimum of R to start analyzing data. In particular, this book is written for readers who have scant experience in computer programming or scripting. I want them to focus on deploying R's rich library of statistical tools to get the desired results rather than being bogged down by the technical details and cryptic syntax of the R language. Although most people think a book like this is about R programming, I prefer to call it **scripting**. The nuance is that the goal of programming is to implement algorithms - a step-by-step procedure to tackle a problem like a cooking recipe. Here we focus on what (expected results) instead of how (algorithms); just tells the R functions here is the data and the parameters, please process them and return the results.

Usage

The book consists of two parts. Part I focuses on the basics. Chapter 1 discusses R objects and operations act on these objects, including descriptive statistical functions. Chapter 2 deals with data wrangling - the process of acquiring, selecting, and organizing data stored in a spreadsheet-like format. Both base R and **tidyverse** data wrangling functions will be discussed. Part I is concluded with data visualization in Chapter 3. A picture is worth a thousand words. The chapter offers guidelines for choosing the most suitable graph according to the intended message and properties of data. Both base R and **ggplot2** graphical systems will be discussed. Part II includes use cases of common statistical tests and intermediate-level R functions that leverage the concepts discussed in Part I.

RStudio

Scripting is best learned through hands-on practice; it is like learning to swim. You don't just listen to lectures. You've got to jump into the water and get wet! To try out the examples and

exercises, you need to use RStudio. There are two options to gain access to RStudio: cloud service (Posit Cloud) and installing RStudio on your own computers. For the first option, no software installation is required. All you need is a Web Browser with Internet connection. For the first time, you can sign up for an account and choose the plan from [Posit Cloud](#). As this book covers the foundational R features, the basic, free plan will work. Your work will be saved in the cloud under your account, regardless what computer you use, as long as you log in to the same account.

Alternatively, if you prefer to have a better control of RStudio, you can install R and RStudio on your own computers. However, it becomes your responsibility to maintain and update them in the future. Why do you need to install two programs instead of just RStudio? In a car analogy, R is like the engine, and RStudio is like the dashboard. RStudio provides a user-friendly Interactive Development Environment (IDE) for users to instruct R to perform the actual work. A favorable feature of R is that it can be installed on a Windows or a Mac computer. If your computer is a Chromebook, Posit Cloud is a preferred, as the installation of R and RStudio on a Chromebook is less straightforward. For Windows and Mac, the installation is simple; it only takes a few double-clicks. Note that you must install R before RStudio. Here are the links to download [R](#) and [RStudio](#).

Lastly, this is a live book that I will continue to update with topics to help readers like you. I invite you to check back periodically to discover new material.

Part I

Part I

1 Basic R

Learning Objectives:

The journey of R scripting starts with a basic understanding of R objects and commands. After finishing this chapter, you should be able to:

- Define R variables.
- Know the differences between an atomic variable and an object variable.
- Store results in variables through the assignment operator.
- Perform arithmetic operations.
- Handle composite objects: vectors, and data frames.
- Define categorical variables.
- Calculate descriptive and summary statistics.
- Utilize inspection functions to figure out basic information of data.
- Format and print the contents of variables to the console.
- Use `seq()` and `with()` functions.

As R scripting is a hands-on topic, I encourage you to open RStudio alongside this book and try out the code to maximize understanding. Let's get started.

1.1 R Variables

An R variable is a named location in the computer's memory that stores some values. Imagine it as a labeled (name) shoebox (memory location) keeping a pair sneakers (value).

Two rules determines how an R variable be named:

1. It must begin with a letter (A-Z or a-z), a dot ".", or an underscore "_", and
2. Subsequent characters can be any combinations of letters, digits (0-9), ".", or "_".

Note that:

- A variable name can be as short as one character to way longer than a normal person wishes (10,000 characters!).
- My suggestion is to give variables meaningful names that can help a bunch for you or someone to comprehend the the code. For instance, in a problem about heart rate, use `zone2` instead of `z` or even worse `x`.
- Be careful that variable names are case-sensitive, e.g., `volume` and `Volume` are different in the eyes of R.
- Avoid naming variables with names that have been reserved by R (<https://rdr.io/r/base/Reserved.html>) or names already used to name R's built-in functions, e.g., `c`, and `head`. For the latter, unfortunately, R is permissive for misnaming variables by remaining silence, costing time to identify the issue. Therefore, the rule of thumb is to exercise self-discipline not to choose R function names as variable names.

1.2 R Objects

Every variable is an R object. In scripting, it is important to know the type(s) of value(s) being stored in a variable, as the types dictate the legitimate operations that act on the variable. Let's begin with data types.

1.2.1 Data Types

R supports six data types:

- Double: decimal numbers, e.g., 5.6, -4.3
- Integer: whole numbers, e.g, -10, 56
- Character: they contain text, e.g., "ACGT", "Hello World"
- Logical: TRUE or FALSE
- Complex*: Complex numbers containing real and imaginary components, e.g., 5+2i
- Raw*: Bare bytes of data.

* We will only discuss the first four types in this book, you can ignore complex and raw types.

1.2.2 Atomic and Composite Variables

From a practical perspective, R variables can be viewed as **atomic variables**, and **composite variables**. An atomic variable contains a single value, and a composite variable consists of a collection of values. Let us begin with two atomic variables: **numeric**, and **character**.

1.2.2.1 Numeric Variables

An atomic numeric variable holds a decimal number (**double**) or a whole number (**integer**). Numbers can be represented in scientific notation. Here are some examples:

```
# Decimal
height <- 5.6

# Whole number (Integer)
sample_size <- 15L

# Scientific notation
avogadro_number <- 6.023e23
```

`<-` is the assignment operator. It looks like a left-pointing arrow, depicting the transfer of value from the right-hand side to the left-hand side. Upon execution, the resultant value of the expression on the right-hand side, if no error, is stored in the named variable on the left-hand side. In RStudio, you can see the variable name and its associated value under the **Environment** tab, which usually occupies the top right quadrant of the RStudio desktop.

Note that R supports another assignment operator: `=`. You can use `<-` and `=` interchangeably. I prefer `<-` to `=`, as `=` can easily confuse with the equality checking operator `==`.

A few words about the scientific notation. It comprises three parts: the coefficient, a constant literal **e**, and the exponent. The **e** refers to the base 10. For example, the R's scientific notation of 6.023×10^{23} is `6.023e23`, where the coefficient and the exponent are 6.023 and 23, respectively. Note that you must not include spaces in a scientific notation, e.g., `6.023e 23`, `6.023 e23`, or `6.023 e 23` will return errors.

Arithmetic Operators

You can apply arithmetic operations on numeric variables. Here are the arithmetic operators: `+` (addition), `-` (subtraction), `*` (multiplication), `/` (division), `**` (exponentiation), and `^` (exponentiation). The last two exponentiation operators perform exactly the same function. Here are some examples:

```
radius <- 2.5
diameter <- 2*pi*radius
pi <- 3.1416
area <- pi*radius**2
volume <- (4/3)*pi*radius^3
```

1.2.2.2 Character Variables

R also support text values, which is in `character` data type. To define a character value, you need to use a pair of quotes to embrace the text. Either a pair of single (') or double quotes (") works. E.g.,

```
course <- 'BIOL265'
```

To figure out whether a variable's type is numeric, character, or something else, use the `typeof()` function,

```
typeof(course)
```

```
[1] "character"
```

```
typeof(area)
```

```
[1] "double"
```

Obviously, it does not make sense to do math with character type. E.g.,

```
course*4
```

```
Error in `course * 4`:
! non-numeric argument to binary operator
```

You can use `nchar()` function to figure out the number of characters, including spaces and punctuations, of a character variable. E.g.,

```
nchar(course)
```

```
[1] 7
```

1.2.3 Vectors

One common composite variable in R is **vectors**. It is an ordered sequence of items that can be processed collectively as a whole, or individually. A vector can be viewed pictorially as a wagon train in which carriages are coupled one after the other. E.g.,

```
# A simple numeric vector for the days in each month
days_of_month <- c(31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31)
```

`days_of_month` is a **vector** object in R. `c()` is the function that **combines or collates** several objects into a vector object. Items are indexed or numbered from 1 to the total number of items in the vector. You can extract a particular item by specifying the index within a pair of square-brackets. E.g.,

```
# Days in February
print(days_of_month[2])
```

```
[1] 28
```

The `[2]` means the 2nd element of a vector. We will discuss more about indexing below.

Here is an example of a character vector:

```
nucleotides <- c('A','C','G','T')
```

Noteworthy that all items held inside a vector must have the same data type. In other words, vectors are always **homogeneous**. For instance, `days_of_month` contains all numeric values, and `nucleotides` contains all character values. If you try to combine objects of different data types into a vector, R will automatically convert them to character type, the lowest denominator of data types. In the example below, even the first and third values are entered as numeric, R coerces them into text "1" and "0".

```
a_vector <- c(1, "success", 0, "fail")
a_vector
```

```
[1] "1"      "success" "0"      "fail"
```

One of the R features that makes arithmetic operations on vectors really convenient is **vectorization**. For instance, if you want to standardize the values of a sample such that the sample mean and sample variance after standardization become 0 and 1, respectively. The formula for standardization is $z_i = \frac{(x_i - \bar{x})}{s}$, where $\bar{x}$ and s are the sample mean and sample standard deviation, respectively. Here's an example

```

heights <- c(5.5, 6.0, 6.1, 5.7, 5.9, 5.6)
x_bar <- mean(heights)
s <- sd(heights)
std_heights <- (heights - x_bar)/s
std_heights

```

```
[1] -1.2677314  0.8451543  1.2677314 -0.4225771  0.4225771 -0.8451543
```

`mean()` and `sd()` are built-in functions that calculate the average and standard deviation of a sample, respectively.

The convenience is that you don't have to specify how to subtract and divide each height individually. Instead, R recognizes the expression `(heights-x_bar)` is to subtract an atomic `x_bar` (scalar) from a vector `heights`. Under the vectorization mechanism, it will subtract `x_bar` from every height in the vector `heights`, similarly for the division by `s`.

`vector` is only one of the many types of objects supported by R. Data frame is another commonly used object types, which will be discussed later.

1.2.4 Factors/Categorical Variables

In R, a **factor** refers to a categorical variable in statistics. A categorical variable is further classified into **nominal** or **ordinal**. A nominal variable has no intrinsic order. Use the `factor()` function to define a categorical variable in R. E.g., a sample of colors is stored in a vector, and you want to define it as a nominal categorical variable.

```

colors <- c("Red","Blue","Blue","Red","Green","Green","Blue","Green","Red","Blue")
colors <- factor(colors, levels=c('Red','Blue','Green'))
colors

```

```
[1] Red  Blue Blue Red  Green Green Blue  Green Red  Blue
Levels: Red Blue Green
```

Categorical variables with intrinsic order are called **ordinal** variables. E.g., the stages of hypertension.

```

patients <- c("Stage 2","Hyperintensive","Stage 2","Hyperintensive","elevated","Stage 2","el
patients <- factor(patients,
                    levels=c('normal','elevated','Stage 1','Stage 2','Hyperintensive'),
                    ordered = TRUE)
patients

```

```
[1] Stage 2          Hyperintensive Stage 2          Hyperintensive elevated
[6] Stage 2          elevated          Stage 1
Levels: normal < elevated < Stage 1 < Stage 2 < Hyperintensive
```

As you can see above, the last line (**Levels**) shows the hypertension levels in ascending order and levels are connected by a less than sign (<).

1.2.5 Data Frames

Besides **vector**, another common R object is **data frame**. Almost all the datasets used for data analysis are organized in data frames. Simply speaking, a data frame is a **tabular** or spreadsheet-like object where values are organized in rows and columns. Each row represents a sample and each column represents a variable. Rows are expected to have equal length, i.e., the same number of columns. If not, R will either complain or pad the missing columns with a special value NA (it stands for not available). Let's take a look at an R's built-in data frame **iris**:

```
# Show the first five rows
head(iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

You can peek into the bottom five rows by the **tail()** function.

```
tail(iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
145	6.7	3.3	5.7	2.5	virginica
146	6.7	3.0	5.2	2.3	virginica
147	6.3	2.5	5.0	1.9	virginica
148	6.5	3.0	5.2	2.0	virginica
149	6.2	3.4	5.4	2.3	virginica
150	5.9	3.0	5.1	1.8	virginica

As you can see above, the `iris` data frame consists of five variables (columns). Here are the R functions to show the shape of a data frame:

```
# the dimension (row x column) of a data frame.  
dim(iris)
```

```
[1] 150   5
```

```
# Only the number of rows  
nrow(iris)
```

```
[1] 150
```

```
# Only the number of columns  
ncol(iris)
```

```
[1] 5
```

So, we know that `iris` contains 150 rows (flower samples) and five columns.

1.2.5.1 Accessing Rows and Columns of a Data Frame

Each element in a data frame can be referenced by row and column indexes like a X-Y Cartesian system. Rows are automatically indexed (numbered), where the 1st row is index 1, the 2nd row is index 2, and the index of the last row is the total number of rows, similarly for columns.

The syntax of accessing individual elements by indexes is `<data frame name>[row, column]`. E.g., the cell at the 5th row and 3rd column of the `iris` is:

```
iris[5,3]
```

```
[1] 1.4
```

If the column index is ignored, all columns of the specified row(s) are returned as shown in Figure 1.1. For example, show all columns of the 3rd row by:

```
# Specify the row number only and leave out the column index.  
iris[3,]
```

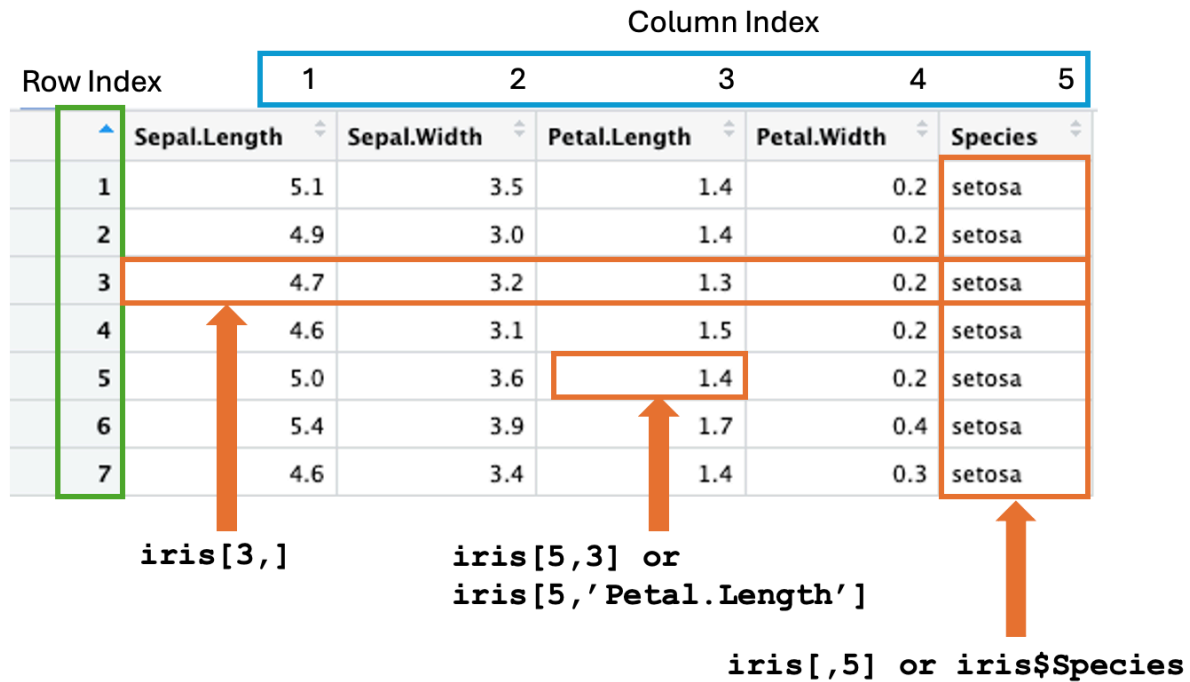


Figure 1.1: Accessing rows and columns of a data frame.

```
Sepal.Length Sepal.Width Petal.Length Petal.Width Species
3          4.7          3.2          1.3          0.2  setosa
```

Similarly, if the row index is skipped, all rows of the specified column(s) are returned. For example,

```
# Specify only the column index.
head(iris[,5])
```

```
[1] setosa setosa setosa setosa setosa setosa
Levels: setosa versicolor virginica
```

Note that the `head()` above is used to limit the number rows displayed by avoiding listing page after page of data rows.

You are not limited to display one row or one column using this indexing scheme. For example, show to 110th and 34th rows and 2nd and 5th columns,


```
iris[c(110,34), c(2,5)]
```

	Sepal.Width	Species
110	3.6	virginica
34	4.2	setosa

As columns are often named ¹, columns can be referenced by their names as well (Figure 1.1). E.g., the `Petal.Length` of the 5th row:

```
iris[5,'Petal.Length']
```

```
[1] 1.4
```

To access more than one column and row, pack the row indexes and column names in vectors by the `c(...)` function, which combines or collates several items into a vector object (see Section 1.2.3). For example, show the `Petal.Length` and `Sepal.Length` of the 123th and 96th rows:

```
iris[c(123,96),c('Petal.Length','Sepal.Length')]
```

	Petal.Length	Sepal.Length
123	6.7	7.7
96	4.2	5.7

If your task is to extract a single column, R provides an alternative way to access a column by its name. The syntax is `<data frame name>$<column name>` ². For example, `iris$Species`, which is equivalent of `iris[,5]`,

```
head(iris$Species)
```

```
[1] setosa setosa setosa setosa setosa setosa
Levels: setosa versicolor virginica
```

We have covered several ways to access rows, columns, or both using indexes and column names. There are additional methods to select rows and columns by conditions. More of this topic will be discussed in Chapter 2.

¹If rows are named, rows can be referenced by their names.

²The `$` method only works for a single column.

1.2.5.2 Create a Data Frame

We can use the `data.frame()` function with the concept of vector to create a dataset in data frame format. For example, a study collected 10 fish with their sex and age. To create the data frame, vectors must be created to represent each variable (column). Here's the R code:

```
sex <- c('F','M','M','F','F','F','M','M','M','M')
age <- c(1,3,2,3,1,1,3,3,1,1)
fish <- data.frame(sex,age)
fish
```

	sex	age
1	F	1
2	M	3
3	M	2
4	F	3
5	F	1
6	F	1
7	M	3
8	M	3
9	M	1
10	M	1

You can use the `summary()` function to examine the summary statistics of the variables.

```
summary(fish)
```

	sex	age
Length	:10	Min. :1.0
N.unique	: 2	1st Qu.:1.0
N.blank	: 0	Median :1.5
Min.nchar	: 1	Mean :1.9
Max.nchar	: 1	3rd Qu.:3.0
		Max. :3.0

Find out more information about `summary()` function below [Section 1.3.3](#).

It is almost done. There is one nuance. In the current form, `sex` is a character variable so its summary statistics are a bit strange. Most often, variable likes that should be categorical/nominal. Therefore, we need to convert `sex` into a `factor` [Section 1.2.4](#). Here's the revised code:

```
sex <- c('F','M','M','F','F','F','M','M','M','M')
sex <- factor(sex, levels=c('F','M'))
age <- c(1,3,2,3,1,1,3,3,1,1)
fish <- data.frame(sex,age)
fish
```

```
      sex age
1      F   1
2      M   3
3      M   2
4      F   3
5      F   1
6      F   1
7      M   3
8      M   3
9      M   1
10     M   1
```

See the differences for `sex` after revision.

```
summary(fish)
```

```
sex      age
F:4  Min.   :1.0
M:6  1st Qu.:1.0
      Median :1.5
      Mean   :1.9
      3rd Qu.:3.0
      Max.   :3.0
```

1.3 Descriptive Statistics

1.3.1 Non-robust Statistics

Sample mean, variance, and standard deviation are non-robust statistics, as they are sensitive to outliers (extreme values).

```
# A sample of pumpkins' weights
pumpkins <- c(6.85,6.56,6.23,7.55,4.45,10.14,3.52,6.62,3.76,7.62)
```

Sample Mean

```
# Sample mean or the average weight  
mean(pumpkins)
```

```
[1] 6.33
```

Sample Variance

```
# Sample variance  
var(pumpkins)
```

```
[1] 4.013489
```

Sample Standard Deviation

```
# Sample standard deviation  
sd(pumpkins)
```

```
[1] 2.003369
```

By the way, the square root of the sample variance $\sqrt{\text{variance}}$ is the standard deviation. See below:

```
sqrt(var(pumpkins))
```

```
[1] 2.003369
```

1.3.2 Robust Statistics

Median and quantile are called robust statistics, as they impede outliers' influence.

Median

```
# A sample of pumpkins' weights  
pumpkins <- c(6.85, 6.56, 6.23, 7.55, 4.45, 10.14, 3.52, 6.62, 3.76, 7.62)
```

```
median(pumpkins)
```

```
[1] 6.59
```

Quantile

The default quantile interval, as the name suggests, is 25%. Note that the R function name is `quantile()`, not `quartile()`, as users can override the default 25% interval.

```
quantile(pumpkins)
```

0%	25%	50%	75%	100%
3.520	4.895	6.590	7.375	10.140

Instead of the default 25% interval, you can specify your preferred quantile(s) and they don't have to be equally divided. Here are some examples:

```
quantile(pumpkins, probs = c(0.1,0.9))
```

10%	90%
3.736	7.872

Or, a specific quantile.

```
quantile(pumpkins, probs = c(0.05))
```

5%
3.628

```
quantile(pumpkins, probs = c(0.95))
```

95%
9.006

Inter-Quartile Region (IQR)

Here is the definition of IQR:

$$IQR = Q3 - Q1$$

```
IQR(pumpkins)
```

```
[1] 2.48
```

IQR is used to determine the upper and lower whiskers in a boxplot. Data points below or above the lower or upper whisker are considered as outliers.

$$\text{Lowerwhisker} = Q1 - 1.5 \times IQR$$

$$\text{Upperwhisker} = Q3 + 1.5 \times IQR$$

1.3.3 summary() Function

A data frame may consists of many variables of different types in various ranges. Therefore, it makes a lot of sense to take a glimpse of all variables before analysis. It can be done by using the `summary()` function to reveal:

1. The variable type of each column.
2. If the column is numeric, a summary of numeric values.
3. If the column is categorical (factor), what are the levels and order, if it is ordinal.

Let's see the summary of `iris`:

```
summary(iris)
```

Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
Min. :4.300	Min. :2.000	Min. :1.000	Min. :0.100
1st Qu.:5.100	1st Qu.:2.800	1st Qu.:1.600	1st Qu.:0.300
Median :5.800	Median :3.000	Median :4.350	Median :1.300
Mean :5.843	Mean :3.057	Mean :3.758	Mean :1.199
3rd Qu.:6.400	3rd Qu.:3.300	3rd Qu.:5.100	3rd Qu.:1.800
Max. :7.900	Max. :4.400	Max. :6.900	Max. :2.500

Species
setosa :50
versicolor:50
virginica :50

The summary will show the range (min. and max.) and quantiles of a numeric column. For a categorical variable (factor), it will show the levels with the number of samples (rows) found for each level, e.g., `Species` is a nominal variable, with 50 samples per species.

1.4 Utility Functions

1.4.1 Inspection Functions

Before diving into data analysis, we often want to know some basic information about it, such as the number of samples, what is the range of a numeric variable, etc. Here we will discuss a few R functions to do that, and we coin them inspection functions.

We will use a sample of pumpkin's weights for illustration.

```
# A sample of pumpkins' weights
pumpkins <- c(6.85,6.56,6.23,7.55,4.45,10.14,3.52,6.62,3.76,7.62)
```

The `length()` shows the number of items in a vector.

```
length(pumpkins)
```

```
[1] 10
```

To be clear, `length()` counts the number of items in an R composite object; the number of items in a vector or the number of columns in a data frame. **If the object is atomic, the function will return 1.** For example, in the following example, `dna` is an atomic, character variable. The function will return 1, not the number of characters.

```
dna <- 'ACGTAGC'
length(dna)
```

```
[1] 1
```

To count the number of characters in a character variable, use the `nchar()` function.

```
nchar(dna)
```

```
[1] 7
```

The `range()` prints the minimum and maximum values of an object ³.

```
range(pumpkins)
```

```
[1] 3.52 10.14
```

If you are interested in either the minimum or maximum, here are the corresponding functions:

```
# The lightest  
min(pumpkins)
```

```
[1] 3.52
```

```
# The heaviest  
max(pumpkins)
```

```
[1] 10.14
```

1.4.2 Printing and Formatting

R provides a function called `print()` that prints text to the console. E.g.,

```
print("Hello World!")
```

```
[1] "Hello World!"
```

```
radius <- 2.5  
diameter <- 2*pi*radius  
print(diameter)
```

```
[1] 15.708
```

Suppose you want to improve the printout by making it more readable, say "The diameter of a circle with radius 2.5 cm is 15.70796." In such a case, you need the format function `paste()`. Here's how to use it:

³Note that `range()` works for non-numeric vector, but we will skip such special operation.


```
radius <- 2.5
diameter <- 2*pi*radius
print(paste("The diameter of a circle with radius",radius,"cm is",diameter,"."))
```

```
[1] "The diameter of a circle with radius 2.5 cm is 15.708 ."
```

The above printout is almost perfect. If you're picky, you don't like the extra space separating the diameter and the period in the above message, you can use `paste()`'s cousin function `paste0()`. The difference between the two is that `paste0()` will not automatically add an extra space between each component given to the function. Below is the previous print statement with `paste0()`:

```
print(paste0("The diameter of a circle with radius",radius,"cm is",diameter,"."))
```

```
[1] "The diameter of a circle with radius2.5cm is15.708."
```

As you can see, there is no more space between the diameter and the period. But it created other issues; there is no space between "The diameter of a circle with radius" and the actual radius as well. So, when you use `paste0()`, you have to take care of the space separation yourselves. Here is the final version for using `paste0()`:

```
print(paste0("The diameter of a circle with radius ",radius," cm is ",diameter,"."))
```

```
[1] "The diameter of a circle with radius 2.5 cm is 15.708."
```

1.4.3 `seq()` - create a series of numbers

There are situations where we need a series of equal-interval numbers. For example, override the default breaks - x-ticks - of a histogram, a vector containing all possible outcomes of tossing two dice, i.e, from 2 to 12.

In R, the `seq()` function generates a list of equal-interval numbers, whole and decimal. `seq()` takes the following parameters:

1. `from` = 1, the default is 1.
2. `to` = 1, the default is 1.
3. `by` = 1, the size of the interval between two adjacent numbers. It takes integer and decimal numbers. If it is negative, the series is decreasing.

4. `length.out` specifies how many numbers are returned. This option is an alternative to `by` when you care only about how many numbers are generated instead of the interval between them. See the example below.
5. `along.with`, it generates the same number of numbers according to what is provided to the `along.with` parameter.

Examples

By default, `seq()` begins with 1 and has an interval of 1. E.g., generate 10 integers starting with 1.

```
seq(10)
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

Starting from and ending with.

```
seq(-10,10)
```

```
[1] -10 -9 -8 -7 -6 -5 -4 -3 -2 -1 0 1 2 3 4 5 6 7 8
[20] 9 10
```

Specify the interval.

```
seq(-10,10, by=2)
```

```
[1] -10 -8 -6 -4 -2 0 2 4 6 8 10
```

A decreasing sequence.

```
seq(10,by=-1)
```

```
[1] 10 9 8 7 6 5 4 3 2 1
```

Just specify the desired numbers to be generated. As you can see, the interval is pretty precise.

```
seq(-5,7,length.out=20)
```

```
[1] -5.00000000 -4.36842105 -3.73684211 -3.10526316 -2.47368421 -1.84210526  
[7] -1.21052632 -0.57894737  0.05263158  0.68421053  1.31578947  1.94736842  
[13]  2.57894737  3.21052632  3.84210526  4.47368421  5.10526316  5.73684211  
[19]  6.36842105  7.00000000
```

Generate a sequence that matches the same number in another sequence.

```
x <- seq(1,5)  
cat("x =",x,"\n")
```

```
x = 1 2 3 4 5
```

```
y <- seq(-5, along.with=x)  
cat("y =",y,"\n")
```

```
y = -5 -4 -3 -2 -1
```

Note: What is the difference between `cat(...)` and `print(paste(...))` you saw above? The main difference is that `print(paste(...))` creates an R object (a text) and then prints it, while `cat(...)` directly writes text to the console without creating a formal R object. See example below you will understand the nuance.

```
msg1 <- print(paste("Hello","World"))
```

```
[1] "Hello World"
```

```
print(msg1)
```

```
[1] "Hello World"
```

```
msg2 <- cat("Hello","World","\n")
```

```
Hello World
```

```
print(msg2)
```

NULL

Is it really important? No, unless you're a R developer.

1.4.4 with()

The function set up the context of a data frame so it spares you from using the `$` to access the columns of a data frame, as it often leads to longer commands, which makes harder to comprehend and troubleshooting. It is also commonly used together with a function applied to one or more columns. E.g., the median of `Petal.Width`. It can be done in two ways:

```
# Use $  
median(iris$Petal.Width)
```

```
[1] 1.3
```

```
# Or, use with()  
with(iris, median(Petal.Width))
```

```
[1] 1.3
```

For the straightforward example above, you do not see the value of `with()`. Let's consider a more complicated row selection example. Find `Sepal.Length` in `iris` where its `Petal.Length` is less than 4 and `Petal.Width` is greater than 1. The `$`-way is:

```
iris$Sepal.Length[iris$Petal.Length<4 & iris$Petal.Width>1]
```

```
[1] 5.2 5.6 5.6 5.5 5.8 5.1
```

Here's the `with()` version that saves typing `iris` repeatedly.

```
with(iris, Sepal.Length[Petal.Length<4 & Petal.Width>1])
```

```
[1] 5.2 5.6 5.6 5.5 5.8 5.1
```

In short, `with()` is English, so it improves the readability of R statements. Second, it saves some typing of the data frame's name. I recommend using `with()` whenever possible.

1.5 Summary

That's a bare minimum of R scripting required for data analysis. In the next chapter, we will focus on operations applied to data frames. Before you leave, it is useful to recap the R functions discussed in this chapter:

Table 1.1: A Summary of R Operators and Functions

<code><-</code>	<code>head()</code>	<code>sd()</code>	<code>+</code>
<code>c()</code>	<code>tail()</code>	<code>range()</code>	<code>-</code>
<code>print()</code>	<code>dim()</code>	<code>min()</code>	<code>*</code>
<code>paste()</code>	<code>nrow()</code>	<code>max()</code>	<code>/</code>
<code>paste0()</code>	<code>ncol()</code>	<code>median()</code>	<code>**</code>
<code>cat()</code>	<code>data.frame()</code>	<code>quantile()</code>	<code>^</code>
<code>typeof()</code>	<code>summary()</code>	<code>IQR()</code>	<code>==</code>
<code>nchar()</code>	<code>mean()</code>	<code>seq()</code>	<code><-</code>
<code>factor()</code>	<code>var()</code>	<code>with()</code>	<code>c()</code>

2 Data Wrangling

Data wrangling refers to the way to view, change, and reorganize rows and columns in a data frame. Before we dive into the operations, it is important to know that there are two distinct schools of data wrangling: base R and **tidyverse**. The base R approach means that no additional packages are required, but the **tidyverse** approach requires the installation of **tidyverse**, even though the installation is very straightforward. In my opinion, **tidyverse** has substantially refined and enhanced data wrangling. You will see it in this chapter. Regardless, we will discuss both approaches. Let's start with the base R version.

Learning Objectives:

This chapter covers common data wrangling techniques. After finishing this chapter, you should be able to:

- Select rows that fulfill specified conditions.
- Randomly sample rows from a data frame.
- Sort rows in a data frame.
- Select a subset of columns.
- Add and remove columns to an existing data frame.
- Tabulate data.

2.1 Base R

2.1.1 Select Rows by Condition(s)

The **subset()** function selects rows if they satisfy the condition(s). Below are six relational operators:

1. **>** greater than.
2. **>=¹** greater than or equal to.

¹No spaces between the two symbols.

3. < less than.
4. <=² less than or equal to.
5. ==³ equal to.
6. !=⁴ not equal to.

If the selection involves multiple conditions, they must be specified in conjunction with relational operators. Here are the three logical operators required for multi-parts conditions:

1. | denotes the logical OR.
2. & denotes the logical AND.
3. ! denotes the logical NOT.

2.1.1.1 subset() with Vectors

Let's define a vector.

```
# A sample of pumpkins' weights
pumpkins <- c(6.85,6.56,6.23,7.55,4.45,10.14,3.52,6.62,3.76,7.62, 7)
```

Select pumpkins heavier than 7 lbs.

```
subset(pumpkins, pumpkins>7)
```

```
[1] 7.55 10.14 7.62
```

Select pumpkins is exactly 7 lbs.

```
subset(pumpkins, pumpkins==7)
```

```
[1] 7
```

Select pumpkins between 6 and 7 lbs, inclusive.

²No spaces between the two symbols.

³No spaces between the two symbols.

⁴No spaces between the two symbols.

```
subset(pumpkins, pumpkins>=6 & pumpkins<=7)
```

```
[1] 6.85 6.56 6.23 6.62 7.00
```

2.1.1.2 subset() with Data Frames

Here is an example that chooses only the `setosa` species from `iris` data frame and keeps them in a new data frame. Note that the equality check is case-sensitive.

```
setosa_iris <- subset(iris, Species == 'setosa')

# How many are selected?
print(paste("There are", nrow(setosa_iris), "rows."))
```

```
[1] "There are 50 rows."
```

```
# Peek into the first few rows of the result.
head(setosa_iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

More than one condition. Select rows with `Sepal.Length` is greater than 5 or `Sepal.Width` is smaller than 3.

```
iris_or_eg <- subset(iris, Sepal.Length > 5 | Sepal.Width < 3 )

# How many?
print(paste("There are", nrow(iris_or_eg), "rows."))
```

```
[1] "There are 124 rows."
```



```
head(iris_or_eg)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
9	4.4	2.9	1.4	0.2	setosa
11	5.4	3.7	1.5	0.2	setosa
15	5.8	4.0	1.2	0.2	setosa
16	5.7	4.4	1.5	0.4	setosa

Select rows with `Petal.Length` is greater than 6 and `Petal.Width` is smaller than 2.

```
# More than one condition. The relational AND operator.
iris_and_eg <- subset(iris, Petal.Length > 6 & Petal.Width < 2 )

# How many?
print(paste("There are", nrow(iris_and_eg), "rows."))
```

```
[1] "There are 2 rows."
```

```
iris_and_eg
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
108	7.3	2.9	6.3	1.8	virginica
131	7.4	2.8	6.1	1.9	virginica

Select everything except for `setosa`.

```
# No setosa iris
iris_no_setosa_eg <- subset(iris, Species != 'setosa')

# How many?
print(paste("There are", nrow(iris_no_setosa_eg), "rows."))
```

```
[1] "There are 100 rows."
```

```
head(iris_no_setosa_eg)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
51	7.0	3.2	4.7	1.4	versicolor
52	6.4	3.2	4.5	1.5	versicolor
53	6.9	3.1	4.9	1.5	versicolor
54	5.5	2.3	4.0	1.3	versicolor
55	6.5	2.8	4.6	1.5	versicolor
56	5.7	2.8	4.5	1.3	versicolor

2.1.2 Random Sampling

It is quite common to randomly sample a subset of items from a vector or rows from a data frame.

2.1.2.1 Sample Items Randomly from a Vector

```
# A sample of pumpkins' weights
pumpkins <- c(6.85,6.56,6.23,7.55,4.45,10.14,3.52,6.62,3.76,7.62, 7)
```

Randomly pick 5 samples without replacement.

```
sample(pumpkins, size=5, replace=F)
```

```
[1] 6.56 6.62 7.00 6.23 3.52
```

The `replace=` option can be twisted to simulate tossing a coin N times. E.g.,

```
sample(c('H','T'), size=10, replace=T)
```

```
[1] "T" "T" "H" "H" "H" "T" "H" "H" "T" "H"
```

2.1.2.2 Sample Rows Randomly from a Data Frame

Let say we want to randomly pick 5 samples from `iris`. It takes to two steps to do that:

Step 1: Create a list (vector) of random row indexes by the `sample()` function. Specify the following parameters:

a.) The range of indexes. In this example, the range is from 1 to the last row of `iris`, which can be obtained from `nrow(iris)`.

- b) How many samples, e.g., `size = 5`.
- c) Can a sample be drawn repeatedly? If not, `replace = F`.

```
rnd_idx <- sample(seq(nrow(iris)), size = 5, replace = F)
rnd_idx
```

```
[1] 25 62 38 16 44
```

`seq(nrow(iris))` generates a vector containing 1, 2, 3, ..., 150.

Step 2: Select rows from `iris` according to the indexes in `rnd_idx`.

```
a_random_sample <- iris[rnd_idx,]
a_random_sample
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
25	4.8	3.4	1.9	0.2	setosa
62	5.9	3.0	4.2	1.5	versicolor
38	4.9	3.6	1.4	0.1	setosa
16	5.7	4.4	1.5	0.4	setosa
44	5.0	3.5	1.6	0.6	setosa

The indexing method is based on the previous chapter. Check out here: [Section 1.2.5.1](#).

2.1.3 Sort Items

2.1.3.1 Sort Items in a Vector

It can be done using the `sort()` function.

```
sort(pumpkins)
```

```
[1] 3.52 3.76 4.45 6.23 6.56 6.62 6.85 7.00 7.55 7.62 10.14
```

Sort it descending order (large to small).

```
sort(pumpkins, decreasing = TRUE)
```

```
[1] 10.14  7.62  7.55  7.00  6.85  6.62  6.56  6.23  4.45  3.76  3.52
```

2.1.3.2 Sort Rows in a Data Frame

Unfortunately, the `sort()` function does not work for the two-dimensional data frame. We have to use the method similar to sampling rows. It takes two steps to sort rows in a certain order. Suppose you want to sort rows in `iris` by `Sepal.Length` in ascending order (small to large).

Step 1: Obtain a list (vector) of row indexes that reflects the sorting order of the data frame.

```
sorted_idx <- order(iris$Sepal.Length)
sorted_idx
```

```
[1] 14  9 39 43 42  4  7 23 48  3 30 12 13 25 31 46  2 10
[19] 35 38 58 107  5  8 26 27 36 41 44 50 61 94  1 18 20 22
[37] 24 40 45 47 99 28 29 33 60 49  6 11 17 21 32 85 34 37
[55] 54 81 82 90 91 65 67 70 89 95 122 16 19 56 80 96 97 100
[73] 114 15 68 83 93 102 115 143 62 71 150 63 79 84 86 120 139 64
[91] 72 74 92 128 135 69 98 127 149 57 73 88 101 104 124 134 137 147
[109] 52 75 112 116 129 133 138 55 105 111 117 148 59 76 66 78 87 109
[127] 125 141 145 146 77 113 144 53 121 140 142 51 103 110 126 130 108 131
[145] 106 118 119 123 136 132
```

Step 2: Rearrange the rows in the data frame according to the sorted row indexes.

```
# The 5 shortest Sepal Lengths
head(iris[sorted_idx,])
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
14	4.3	3.0	1.1	0.1	setosa
9	4.4	2.9	1.4	0.2	setosa
39	4.4	3.0	1.3	0.2	setosa
43	4.4	3.2	1.3	0.2	setosa
42	4.5	2.3	1.3	0.3	setosa
4	4.6	3.1	1.5	0.2	setosa

```
# The 5 longest Sepal Lengths
tail(iris[sorted_idx,])
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
106	7.6	3.0	6.6	2.1	virginica
118	7.7	3.8	6.7	2.2	virginica
119	7.7	2.6	6.9	2.3	virginica
123	7.7	2.8	6.7	2.0	virginica
136	7.7	3.0	6.1	2.3	virginica
132	7.9	3.8	6.4	2.0	virginica

Note that the steps above do NOT change the original order of the rows in the data frame. If you intend to change the row order permanently, use the assignment operator to update the existing data frame or create a new data frame to store the sorted rows. See below:

```
ascending_iris <- iris[sorted_idx,]

# The shortest Sepal Length
head(ascending_iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
14	4.3	3.0	1.1	0.1	setosa
9	4.4	2.9	1.4	0.2	setosa
39	4.4	3.0	1.3	0.2	setosa
43	4.4	3.2	1.3	0.2	setosa
42	4.5	2.3	1.3	0.3	setosa
4	4.6	3.1	1.5	0.2	setosa

```
# The longest Sepal Length
tail(ascending_iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
106	7.6	3.0	6.6	2.1	virginica
118	7.7	3.8	6.7	2.2	virginica
119	7.7	2.6	6.9	2.3	virginica
123	7.7	2.8	6.7	2.0	virginica
136	7.7	3.0	6.1	2.3	virginica
132	7.9	3.8	6.4	2.0	virginica

To sort rows in descending order (large to small), add the parameter `decreasing = T` to the `order()` function.

```
desc_idx <- order(iris$Sepal.Length, decreasing = T)
desc_idx
```

```
[1] 132 118 119 123 136 106 131 108 110 126 130 103  51  53 121 140 142  77
[19] 113 144  66  78  87 109 125 141 145 146  59  76  55 105 111 117 148  52
[37]  75 112 116 129 133 138  57  73  88 101 104 124 134 137 147  69  98 127
[55] 149  64  72  74  92 128 135  63  79  84  86 120 139  62  71 150  15  68
[73]  83  93 102 115 143  16  19  56  80  96  97 100 114  65  67  70  89  95
[91] 122  34  37  54  81  82  90  91  6  11  17  21  32  85  49  28  29  33
[109]  60  1  18  20  22  24  40  45  47  99  5  8  26  27  36  41  44  50
[127]  61  94  2  10  35  38  58 107  12  13  25  31  46  3  30  4  7  23
[145]  48  42  9  39  43  14
```

```
decreasing_iris <- iris[desc_idx,]
head(decreasing_iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
132	7.9	3.8	6.4	2.0	virginica
118	7.7	3.8	6.7	2.2	virginica
119	7.7	2.6	6.9	2.3	virginica
123	7.7	2.8	6.7	2.0	virginica
136	7.7	3.0	6.1	2.3	virginica
106	7.6	3.0	6.6	2.1	virginica

```
tail(decreasing_iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
48	4.6	3.2	1.4	0.2	setosa
42	4.5	2.3	1.3	0.3	setosa
9	4.4	2.9	1.4	0.2	setosa
39	4.4	3.0	1.3	0.2	setosa
43	4.4	3.2	1.3	0.2	setosa
14	4.3	3.0	1.1	0.1	setosa

2.1.4 Rearranging Columns

Suppose you want to move the `Species` column to the leftmost position. It can be done in two ways.

```
# Reorder columns by name
new_iris <- iris[,c("Species", "Sepal.Length", "Sepal.Width", "Petal.Length", "Petal.Width")]
head(new_iris)
```

	Species	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
1	setosa	5.1	3.5	1.4	0.2
2	setosa	4.9	3.0	1.4	0.2
3	setosa	4.7	3.2	1.3	0.2
4	setosa	4.6	3.1	1.5	0.2
5	setosa	5.0	3.6	1.4	0.2
6	setosa	5.4	3.9	1.7	0.4

```
# Reorder columns by index
new_iris <- iris[,c(5,1,2,3,4)]
head(new_iris)
```

	Species	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
1	setosa	5.1	3.5	1.4	0.2
2	setosa	4.9	3.0	1.4	0.2
3	setosa	4.7	3.2	1.3	0.2
4	setosa	4.6	3.1	1.5	0.2
5	setosa	5.0	3.6	1.4	0.2
6	setosa	5.4	3.9	1.7	0.4

Note that if a data frame contains many columns, say >100, this method is daunting. A much better way can be done using `tidyverse`, so stay tuned.

2.1.5 Focus on a Subset of Columns

If a data frame contains many columns, way more than you need for analysis. It is neat to focus on those columns and leave out the rest. Let's suppose you want to focus only on sepal information and species.

```
sepal_iris <- iris[,c("Sepal.Length", "Sepal.Width", "Species")]
dim(sepal_iris)
```

```
[1] 150 3
```

```
head(sepal_iris)
```

	Sepal.Length	Sepal.Width	Species
1	5.1	3.5	setosa
2	4.9	3.0	setosa
3	4.7	3.2	setosa
4	4.6	3.1	setosa
5	5.0	3.6	setosa
6	5.4	3.9	setosa

2.1.6 Add a New Column

Suppose we want to add a new column called `Sepal.Ratio`, which is the ratio `Sepal.Length` to `Sepal.Width`. Here is the code:

```
iris$Sepal.Ratio <- iris$Sepal.Length/iris$Sepal.Width  
head(iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species	Sepal.Ratio
1	5.1	3.5	1.4	0.2	setosa	1.457143
2	4.9	3.0	1.4	0.2	setosa	1.633333
3	4.7	3.2	1.3	0.2	setosa	1.468750
4	4.6	3.1	1.5	0.2	setosa	1.483871
5	5.0	3.6	1.4	0.2	setosa	1.388889
6	5.4	3.9	1.7	0.4	setosa	1.384615

You can see the new column `Sepal.Ratio` is added to the rightmost position.

Suppose you want to add a new column that records the name of the researcher who collected the sample. If it is done by one researcher, you don't have to repeat the same name in a vector. Here is the code:

```
iris$Researcher <- "Brian"  
head(iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species	Sepal.Ratio
1	5.1	3.5	1.4	0.2	setosa	1.457143
2	4.9	3.0	1.4	0.2	setosa	1.633333

3	4.7	3.2	1.3	0.2	setosa	1.468750
4	4.6	3.1	1.5	0.2	setosa	1.483871
5	5.0	3.6	1.4	0.2	setosa	1.388889
6	5.4	3.9	1.7	0.4	setosa	1.384615

	Researcher
1	Brian
2	Brian
3	Brian
4	Brian
5	Brian
6	Brian

2.1.7 Remove a Column

You can remove a column from a data frame, e.g., remove the `Sepal.Length` and `Researcher` added above.

```
iris$Sepal.Length <- NULL
iris$Researcher <- NULL

# the columns are gone
head(iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

2.1.8 Tabulation

Tally the number of samples per one or more categorical variables (factors) by the `table()` function.

```
table(iris$Species)
```

setosa	versicolor	virginica
50	50	50

```
# Alternatively, use with()
with(iris, table(Species))
```

```
Species
      setosa versicolor  virginica
       50         50         50
```

The data frame can be tallied by more than one categorical variable. Let's use another built-in dataset C02 to illustrate the point.

```
with(C02, table(Plant, Type))
```

```
      Type
Plant Quebec Mississippi
Qn1       7             0
Qn2       7             0
Qn3       7             0
Qc1       7             0
Qc3       7             0
Qc2       7             0
Mn3       0             7
Mn2       0             7
Mn1       0             7
Mc2       0             7
Mc3       0             7
Mc1       0             7
```

That's all for data wrangling using base R.

2.2 tidyverse

tidyverse is an alternative, and importantly, less cryptic way for performing data wrangling of data frames. One design goal of is to name functions using **verbs**, which improves code readability, setting it apart from using unpronounceable symbols, such as \$, [, etc. by base R as shown in the previous section. **However, tidyverse works with data frames only.** You still need the base R to process vectors.

Before we dive into **tidyverse**, it is noteworthy to clarify that **tidyverse** is an integrated ecosystem of R packages designed for data science. The data wrangling functions discuss below

are actually provided by two packages: `dplyr`, and `tidyr`. As this book is a little book, you can simply assume everything provided by the umbrella package `tidyverse`.

As `tidyverse` is an add-on package, you have to install it manually. But the installation is simple. In RStudio, click the menu option *Tools*, select the first option *Install Packages*. At the *Packages* box, type “tidyverse”, and click the “Install” button. If you only need to install it once until after a major R upgrade.

2.2.1 Activate tidyverse

After opening a new session, you need to activate `tidyverse` using the `require()` function.

```
require(tidyverse)
```

Loading required package: tidyverse

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.2
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

The `require()` function has churned out messages telling you what packages (9 of them) are loaded together.

Alternatively, you can use the `library()` function to activate a package. Putting aside technical nuances, these two functions are largely doing the same thing. Since activation is an action, using a verb (`require()`) makes more sense than using a noun (`library()`). Note that you only need to activate a package once per R session.

Let’s use `tidyverse` to repeat the data wrangling tasks by base R above.

2.2.2 Select Rows by Condition(s)

The function is `filter()`. It chooses rows that satisfy the specified condition(s). The syntax is `filter(data_frame, conditions)`. E.g.,

```
# select all setosa iris
setosa_iris <- filter(iris, Species == 'setosa')

# How many?
print(paste("There are", nrow(setosa_iris), "rows."))
```

```
[1] "There are 50 rows."
```

```
# More than one condition. The symbol for the logical OR operator is a vertical bar |
iris_or_eg <- filter(iris, Sepal.Length > 5 | Sepal.Width < 3 )

# How many?
print(paste("There are", nrow(iris_or_eg), "rows."))
```

```
[1] "There are 124 rows."
```

```
head(iris_or_eg)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	5.4	3.9	1.7	0.4	setosa
3	4.4	2.9	1.4	0.2	setosa
4	5.4	3.7	1.5	0.2	setosa
5	5.8	4.0	1.2	0.2	setosa
6	5.7	4.4	1.5	0.4	setosa

Specify more than one condition. Use a relational operator to join conditions together.

```
iris_and_eg <- filter(iris, Petal.Length > 6 & Petal.Width < 2 )

# How many?
print(paste("There are", nrow(iris_and_eg), "rows."))
```

```
[1] "There are 2 rows."
```

```
iris_and_eg
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	7.3	2.9	6.3	1.8	virginica
2	7.4	2.8	6.1	1.9	virginica

```
# No setosa iris
iris_not_eg <- filter(iris, Species != 'setosa')

# How many?
print(paste("There are", nrow(iris_not_eg), "rows."))
```

```
[1] "There are 100 rows."
```

```
head(iris_not_eg)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	7.0	3.2	4.7	1.4	versicolor
2	6.4	3.2	4.5	1.5	versicolor
3	6.9	3.1	4.9	1.5	versicolor
4	5.5	2.3	4.0	1.3	versicolor
5	6.5	2.8	4.6	1.5	versicolor
6	5.7	2.8	4.5	1.3	versicolor

2.2.3 Select Rows Randomly

With tidyverse, you can do it in one step by using the `sample_n()` function.

```
a_random_sample <- sample_n(iris, size = 5)

a_random_sample
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	4.8	3.4	1.9	0.2	setosa
2	6.7	3.1	4.4	1.4	versicolor
3	5.8	2.7	4.1	1.0	versicolor
4	5.7	2.8	4.1	1.3	versicolor
5	6.1	2.9	4.7	1.4	versicolor

2.2.4 Sort Rows in a Data Frame

Use `arrange()`. Note that the function will not change the original data frame. Use the assignment operator (`<-`) to make permanent changes to the original data frame or store the changes in a new data frame. For example, sort `iris` by `Sepal.Length`, and store the sorted result in a new data frame.

```
sorted_iris <- arrange(iris, Sepal.Length)

# The shortest Sepal Length
head(sorted_iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	4.3	3.0	1.1	0.1	setosa
2	4.4	2.9	1.4	0.2	setosa
3	4.4	3.0	1.3	0.2	setosa
4	4.4	3.2	1.3	0.2	setosa
5	4.5	2.3	1.3	0.3	setosa
6	4.6	3.1	1.5	0.2	setosa

```
# The longest Sepal Length
tail(sorted_iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
145	7.6	3.0	6.6	2.1	virginica
146	7.7	3.8	6.7	2.2	virginica
147	7.7	2.6	6.9	2.3	virginica
148	7.7	2.8	6.7	2.0	virginica
149	7.7	3.0	6.1	2.3	virginica
150	7.9	3.8	6.4	2.0	virginica

To sort rows in descending order, simply mark the sorting column with a minus `-`. E.g.,

```
decreasing_iris <- arrange(iris, -Sepal.Length)
head(decreasing_iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	7.9	3.8	6.4	2.0	virginica
2	7.7	3.8	6.7	2.2	virginica
3	7.7	2.6	6.9	2.3	virginica

4	7.7	2.8	6.7	2.0	virginica
5	7.7	3.0	6.1	2.3	virginica
6	7.6	3.0	6.6	2.1	virginica

```
tail(decreasing_iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
145	4.6	3.2	1.4	0.2	setosa
146	4.5	2.3	1.3	0.3	setosa
147	4.4	2.9	1.4	0.2	setosa
148	4.4	3.0	1.3	0.2	setosa
149	4.4	3.2	1.3	0.2	setosa
150	4.3	3.0	1.1	0.1	setosa

Specify more than one sorting column is much easier in tidyverse than base R. E.g. sort iris in descending order of `Sepal.Length` and ascending order of `Petal.Length`.

```
double_sorted_iris <- arrange(iris, -Sepal.Length, Petal.Length)
head(double_sorted_iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	7.9	3.8	6.4	2.0	virginica
2	7.7	3.0	6.1	2.3	virginica
3	7.7	3.8	6.7	2.2	virginica
4	7.7	2.8	6.7	2.0	virginica
5	7.7	2.6	6.9	2.3	virginica
6	7.6	3.0	6.6	2.1	virginica

```
tail(double_sorted_iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
145	4.6	3.1	1.5	0.2	setosa
146	4.5	2.3	1.3	0.3	setosa
147	4.4	3.0	1.3	0.2	setosa
148	4.4	3.2	1.3	0.2	setosa
149	4.4	2.9	1.4	0.2	setosa
150	4.3	3.0	1.1	0.1	setosa

For rows with equal `Sepal.Length` (rows 2-5, 147-149), you can see their `Petal.Length` are in ascending order.

2.2.5 Reorganize columns of a data frame

Suppose you want to move the `Species` column to the leftmost position. You're going to love `tidyverse`, as it can be done easily by using two functions: `select()` and `everything()`.

```
# Reorder columns by name
new_iris <- select(iris, Species, everything())
head(new_iris)
```

	Species	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
1	setosa	5.1	3.5	1.4	0.2
2	setosa	4.9	3.0	1.4	0.2
3	setosa	4.7	3.2	1.3	0.2
4	setosa	4.6	3.1	1.5	0.2
5	setosa	5.0	3.6	1.4	0.2
6	setosa	5.4	3.9	1.7	0.4

2.2.6 Focus on a Subset of Columns

If a data frame contains many columns, way more than you need for analysis. It is neat to focus on those columns and remove the rest. E.g., use the `select()` function to keep only sepal information and species.

```
sepal_iris <- select(iris, Sepal.Length, Sepal.Width, Species)
dim(sepal_iris)
```

```
[1] 150  3
```

```
head(sepal_iris)
```

	Sepal.Length	Sepal.Width	Species
1	5.1	3.5	setosa
2	4.9	3.0	setosa
3	4.7	3.2	setosa
4	4.6	3.1	setosa
5	5.0	3.6	setosa
6	5.4	3.9	setosa

2.2.7 Add a New Column

Use the `mutate()` function to add columns to a data frame.

```
iris <- mutate(iris, Sepal.Ratio = Sepal.Length/Sepal.Width)
head(iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species	Sepal.Ratio
1	5.1	3.5	1.4	0.2	setosa	1.457143
2	4.9	3.0	1.4	0.2	setosa	1.633333
3	4.7	3.2	1.3	0.2	setosa	1.468750
4	4.6	3.1	1.5	0.2	setosa	1.483871
5	5.0	3.6	1.4	0.2	setosa	1.388889
6	5.4	3.9	1.7	0.4	setosa	1.384615

2.2.8 Remove a Column

Use the `select()` function and put a minus '-' symbol before the column name(s) to be removed.

```
iris <- select(iris, -Sepal.Ratio)
head(iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

2.2.9 Rename a Column

So far, you have seen all the data wrangling tasks performed by base R using `tidyverse`. Starting from here, you are going to demonstrate tasks that are harder to do with base R. The first task is to rename a column. Suppose you want to rename `Species` to `Iris_Species`. You can do it by the `rename()` function:

```
# new_name = existing_name
iris <- rename(iris, Iris_Species = Species)
head(iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Iris_Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

Let's revert the renaming so it will not affect the following examples.

```
iris <- rename(iris, Species = Iris_Species)
head(iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

2.2.10 Pipe

Often, a data wrangling is a multi-step process in which many intermediate steps are involved before producing the final result. E.g., select iris where its `Petal.Length` is between 0.5 and 2, and sort the results in descending order of `Petal.Length`. As you can see, it involves filtering, followed by sorting. It can be done into two steps via an intermediate data frame as below:

```
tmp_iris <- filter(iris, Petal.Length >= 0.5 & Petal.Length <=2)
final_iris <- arrange(tmp_iris, -Petal.Length)
dim(final_iris)
```

```
[1] 50  5
```

```
head(final_iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	4.8	3.4	1.9	0.2	setosa
2	5.1	3.8	1.9	0.4	setosa
3	5.4	3.9	1.7	0.4	setosa

4	5.7	3.8	1.7	0.3	setosa
5	5.4	3.4	1.7	0.2	setosa
6	5.1	3.3	1.7	0.5	setosa

The final result contains 50 rows.

tidyverse provides a neater way in which no intermediate data frames are produced. What is new is the **pipe** concept. You can imagine a pipe is like the assembly line of a factory, where an upstream work station passes (pipe) intermediate results to the next work station for further processing.

The pipe symbol is represented by `%>%`, no space between them. Let's see how to apply the pipe to the above example without creating the intermediate data frame `tmp_iris`.

```
final_results <- iris %>%
  filter(Petal.Length >= 0.5 & Petal.Length <=2) %>%
  arrange(-Petal.Length)
dim(final_results)
```

```
[1] 50  5
```

```
head(final_results)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	4.8	3.4	1.9	0.2	setosa
2	5.1	3.8	1.9	0.4	setosa
3	5.4	3.9	1.7	0.4	setosa
4	5.7	3.8	1.7	0.3	setosa
5	5.4	3.4	1.7	0.2	setosa
6	5.1	3.3	1.7	0.5	setosa

Let's walk through the code above line by line.

- `iris %>%` sends all rows, 150 in total, to the downstream `filter()` function.
- `filter(Petal.Length >= 0.5 & Petal.Length <=2) %>%` receives the data, and check each row if it satisfies the specified conditions. If so, send it to the downstream process; otherwise do not pass down.
- `arrange(-Petal.Length)` received rows that satisfy the previous filtering process, and sort them in descending order of `Petal.Length`.
- `final_results <-` the sorted result is assigned to the variable `final_results`.

By using pipe (`%>%`), you can avoid intermediate variables, making the code cleaner and easily to read.

2.2.11 Tabulation and Summarize

Another task which is useful but difficult to get it done in base R is to produce summary information by group. E.g., you want to produce a table that shows the average sepal length for each species. Here's how to do it using two new functions `group_by()` and `summarize()`, and pipe them together.

E.g., tally the number of samples by `Plant` and 'Type' in `C02`. The function that does the tallying is `n()`.

```
C02 %>%  
  group_by(Plant, Type) %>%  
  summarize(n=n())
```

```
`summarise()` has regrouped the output.  
i Summaries were computed grouped by Plant and Type.  
i Output is grouped by Plant.  
i Use `summarise(.groups = "drop_last")` to silence this message.  
i Use `summarise(.by = c(Plant, Type))` for per-operation grouping  
  (`?dplyr::dplyr_by`) instead.
```

```
# A tibble: 12 x 3  
# Groups:   Plant [12]  
  Plant Type      n  
  <ord> <fct>    <int>  
1 Qn1   Quebec      7  
2 Qn2   Quebec      7  
3 Qn3   Quebec      7  
4 Qc1   Quebec      7  
5 Qc3   Quebec      7  
6 Qc2   Quebec      7  
7 Mn3   Mississippi  7  
8 Mn2   Mississippi  7  
9 Mn1   Mississippi  7  
10 Mc2   Mississippi  7  
11 Mc3   Mississippi  7  
12 Mc1   Mississippi  7
```

You do not see much difference between base R and `tidyverse` in the example above. But if you want to do more than tallying, `summarize()` is a treat, as `summarize()` supports the descriptive statistics functions mentioned before, such as `mean()`, `sd()`, `median()`, `quantile()`, etc. E.g,

```
iris %>%
  group_by(Species) %>%
  summarize(u=mean(Sepal.Length))
```

```
# A tibble: 3 x 2
  Species      u
  <fct>      <dbl>
1 setosa    5.01
2 versicolor 5.94
3 virginica 6.59
```

You can apply multiple functions in `summarize()`. E.g., shows the number of samples and standard deviation.

```
iris %>%
  group_by(Species) %>%
  summarize(n=n(),
            u=mean(Sepal.Length),
            s=sd(Sepal.Length))
```

```
# A tibble: 3 x 4
  Species      n      u      s
  <fct>    <int> <dbl> <dbl>
1 setosa     50  5.01 0.352
2 versicolor  50  5.94 0.516
3 virginica   50  6.59 0.636
```

You can group by more than one column. Let's switch to the `C02` dataset to illustrate this point.

```
C02 %>%
  group_by(Plant, Type, Treatment) %>%
  summarize(n=n(),
            u=mean(conc),
            s=sd(conc))
```

``summarise()`` has regrouped the output.

i Summaries were computed grouped by Plant, Type, and Treatment.

i Output is grouped by Plant and Type.

i Use ``summarise(.groups = "drop_last")`` to silence this message.

i Use ``summarise(.by = c(Plant, Type, Treatment))`` for per-operation grouping (``?dplyr::dplyr_by``) instead.

```
# A tibble: 12 x 6
# Groups:   Plant, Type [12]
  Plant Type      Treatment      n      u      s
  <ord> <fct>      <fct>    <int> <dbl> <dbl>
1 Qn1   Quebec    nonchilled      7    435   318.
2 Qn2   Quebec    nonchilled      7    435   318.
3 Qn3   Quebec    nonchilled      7    435   318.
4 Qc1   Quebec     chilled      7    435   318.
5 Qc3   Quebec     chilled      7    435   318.
6 Qc2   Quebec     chilled      7    435   318.
7 Mn3   Mississippi nonchilled      7    435   318.
8 Mn2   Mississippi nonchilled      7    435   318.
9 Mn1   Mississippi nonchilled      7    435   318.
10 Mc2   Mississippi chilled      7    435   318.
11 Mc3   Mississippi chilled      7    435   318.
12 Mc1   Mississippi chilled      7    435   318.
```

Two remarks:

1. `group_by()` is never used alone. It is often followed by `summarize()`.
2. The variables feed to `group_by()` must be factor, i.e., categorical.

2.2.12 A Summary of tidyverse functions

- `filter()`: select rows that satisfy the specified conditions.
- `select()`: choose or drop certain columns.
- `arrange()`: sort rows by the specified columns.
- `rename()`: change the name of a column.
- `mutate()`: add a column or modify the values of an existing column.
- `group_by()`, followed by `summarize()`
- pipe `%>%`: funnel intermediate result from an upstream process to the input of an immediate downstream process.

2.3 Summary

In this chapter, you have seen how to perform data wrangling with basic R and `tidyverse`. What you have seen is only a fraction of what `tidyverse` can do. Readers who want to explore the full capability of `tidyverse` should consult the most authoritative source: [R for Data Science](#) by Hadley Wickham & Garrett Golemund. In the next chapter, you will see how to plot data. A picture is worth a thousand words. Data visualization is an indispensable step for data analysis. But before the next topic, let's recall all the new R commands discussed in this chapter.

Table 2.1: A Summary of R Operators and Functions

	<code>sample()</code>	<code>filter()</code>	<code>summarize()</code>
&	<code>order()</code>	<code>select()</code>	<code>n()</code>
==	NULL (a keyword)	<code>arrange()</code>	pipe <code>%>%</code>
!	<code>table()</code>	<code>rename()</code>	
!=	<code>library()</code>	<code>mutate()</code>	
<code>subset()</code>	<code>require()</code>	<code>group_by()</code>	

3 Data Visualization

Now you have cleaned, organized, and analyzed the data, the next step is to share the findings and insights with the audience, raising their attention to the study. Row and columns of numbers may not be palatable to most people. To assist the audience to unpack the message embedded in the data, a graph is recommended. In this chapter, we will discuss various types of graphs, what they are good at and not so good at, and how to make them in R using two different graphical systems.

Learning Objectives:

After finishing this chapter, you should be able to:

- Choose the most appropriate graph based on the number of variables and their types.
- Convey the message you want to convey to the audience embedded in the data through graphs.
- Decorate graphs with labels, a main title, color, and other graphical attributes using the base R graphical system.
- Create publication-quality graphs using the `ggplot2` package.

3.1 Types of Graphs

The best way to choose the most appropriate graph depends on the data. Specifically, the types of variables (numeric or categorical) and how many of them. Here, you will plot graphs that entail:

1. One numeric variable.
2. One categorical variable.
3. One categorical and one numeric variables.
4. Two or more numeric variables.
5. A summary table.

3.2 Base R Graphics

This graphical system is built into the base R; no additional package is required. It is simple, ideal for creating graphs promptly for prototyping purposes and get the job done. The flip side is that the graphs may be bare-bones and unrefined. Some features are hard to be implemented. There is no one graphical system for all data visualization tasks. It is highly recommended to master both systems. Let's dive into each graph one by one.

3.2.1 One Numeric Variable

If you want to examine the distribution of a numeric variable (such as the spread, peak(s), symmetry, and flatness), a histogram is the choice. On the other hand, if your concern is robust statistics (median, outliers, etc.), a box plot will do the job. Let's start with plotting a histogram.

3.2.1.1 Histogram

Let's examine the distribution of `iris`'s `Sepal.Width`, a numeric variable, using a histogram. Below are the codes for creating four histograms that display the same information but with different refinements.

```
par(mfrow=c(2,2))

# Top left
hist(iris$Sepal.Width)

# Top right
hist(iris$Sepal.Width,
     xlab="Sepal Width",
     ylab = "Count",
     main = "Histogram of Sepal Width")

# Bottom left
hist(iris$Sepal.Width,
     col = 'skyblue',
     border = 'white',
     xlab="Sepal Width",
     ylab = "Count",
     main = "Color options")

# Bottom right
```

```
hist(iris$Sepal.Width,
     breaks = seq(2,4.5,by=0.1),
     col = 'skyblue',
     border = 'white',
     xlab="Sepal Width",
     ylab = "Count",
     main = "breaks option")
```

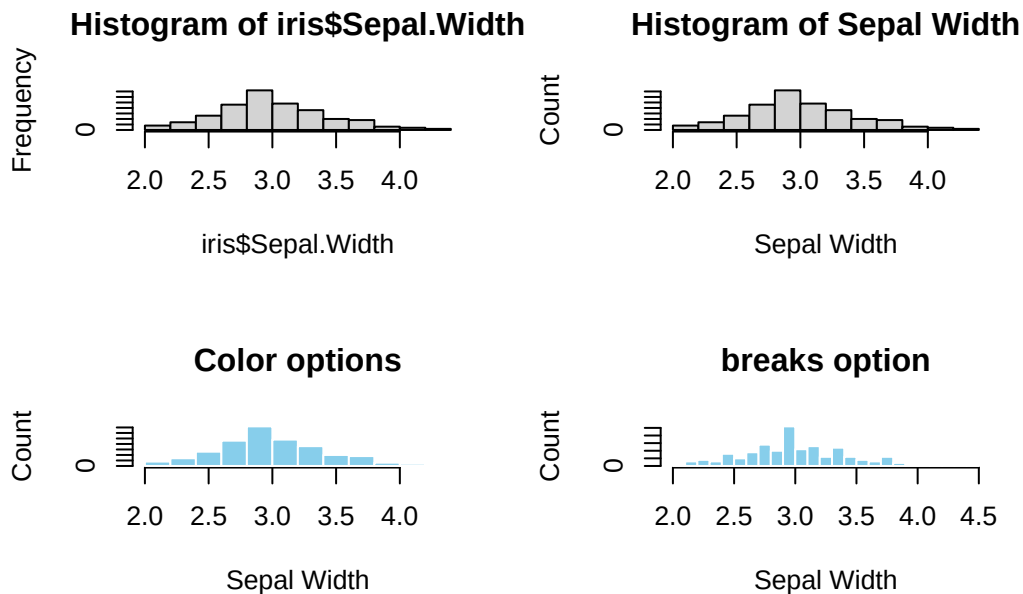


Figure 3.1: Base R Histogram. Top left: default. Top right: Customized X-Y labels and the main title. Bottom left: change color of bars and border. Bottom right: change the bin width (`breaks=`)

For the bottom-right graph, check out this line: `breaks = seq(2,4.5,by=0.1)`. The aim is to customize the width of the bars. The `seq()` function (Section 1.4.3) generates a list of numbers (2.0, 2.1 ... 4.4, 4.5) that represents the x-ticks.

Note: `par(mfrow=c(2,2))` splits the plotting area into a 2x2 grid such that multiple graphs can be displayed together. If you plot contains only one graph, it can be ignored.

3.2.1.2 Box Plot

If robust statistics of the data is the message, the most appropriate graph is a box plot. In a box plot, the y-axis represents the numeric variable, and the x-axis has no meaning.

```
par(mfrow=c(2,2))

# Top left
boxplot(iris$Sepal.Width)

# Top right
boxplot(iris$Sepal.Width,
        ylab="Sepal Width",
        main="Box plot of Sepal Width")

# Bottom left
boxplot(iris$Sepal.Width,
        col = "skyblue",
        ylab="Sepal Width",
        outcol = 'red',
        outpch = 19,
        main="Color options")

# Bottom right
boxplot(iris$Sepal.Width,
        horizontal = TRUE,
        xlab="Sepal Width",
        main="Horizontal Box plot")
```

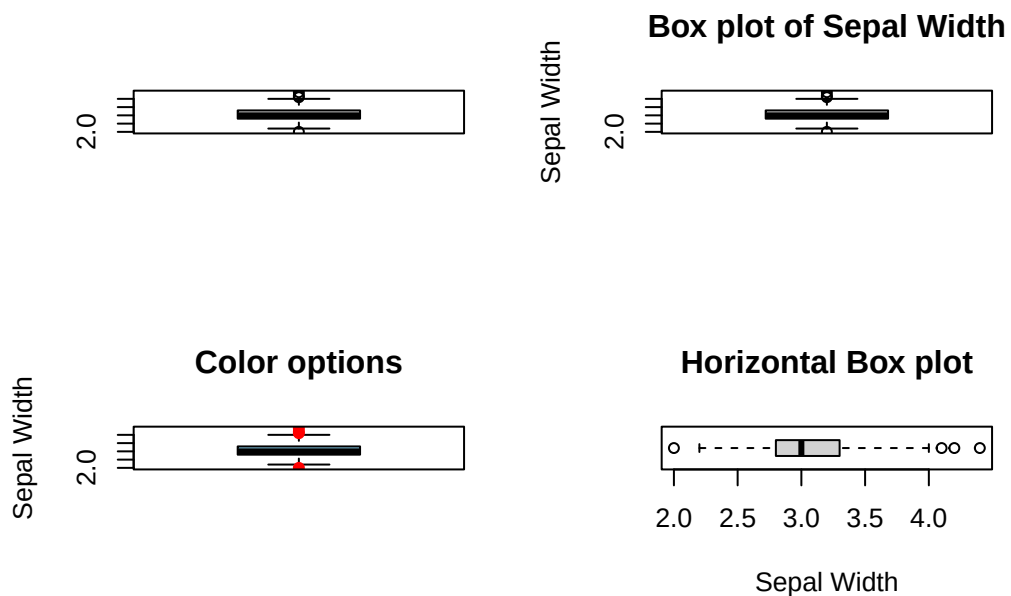


Figure 3.2: Base R Box plot. Top left: Default, Top right: labels and main title. Bottom left: color options. Bottom right: `horizontal = TRUE`

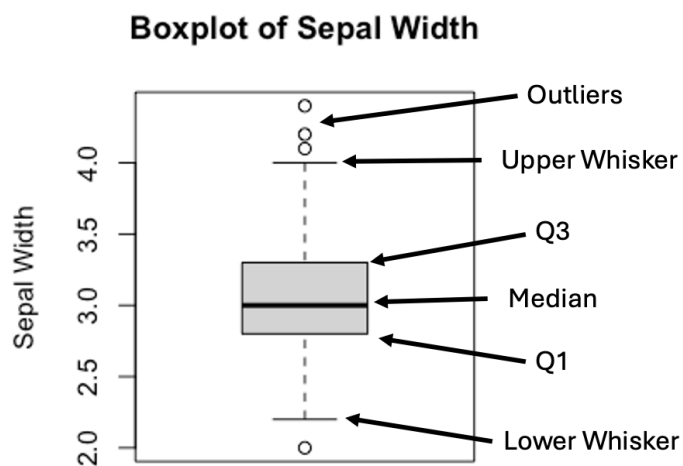


Figure 3.3: Anatomy of a Box plot

The upper and lower whiskers are defined as:

$$Upper\ Whisker = Q3 + 1.5 \times IQR$$

$$\text{Lower Whisker} = Q1 - 1.5 \times IQR$$

, where $IQR = (Q3 - Q1)$.

As a practice, data points above the upper whisker or below the lower whiskers are labeled as outliers.

While a box plot summarizes data into quartiles, a histogram portrays the underlying distribution—such as peaks and symmetry—that is otherwise difficult to spot. Is it a great idea to combine the best of both plots in a single graph? To ensure that the combined plot makes sense, we need to ensure that the axes of the two plots are in the same range. Here's an example below:

```
par(mfrow=c(2,1))

hist(iris$Sepal.Width,
     breaks = seq(2,4.5,by=0.1),
     col = 'skyblue',
     border = 'white',
     xlim = c(2,4.5),
     xlab = "",
     ylab = "Count",
     main = "Stacking Up Graphs")

boxplot(iris$Sepal.Width,
        horizontal = TRUE,
        ylim = c(2,4.5),
        xlab="Sepal Width",
        main="")
```

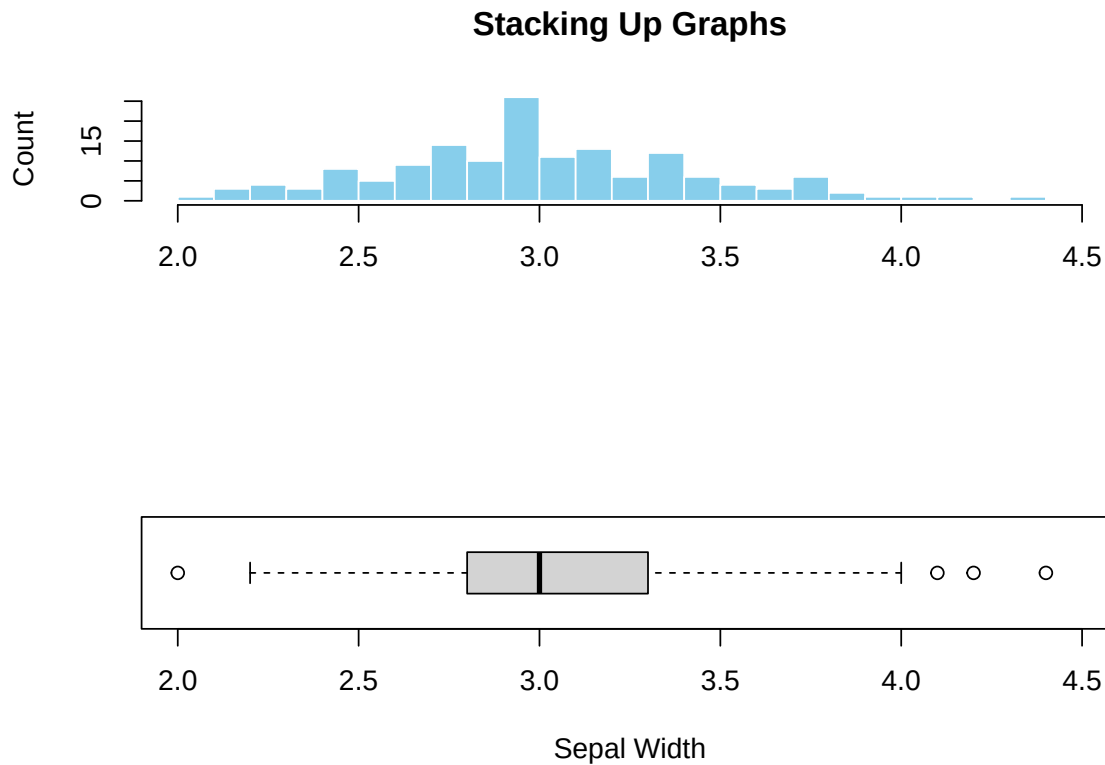


Figure 3.4: Stacking up a histogram and a box plot.

The `ylim` of the histogram and the `xlim` of the box plot are the specific parameters used to align the two graphs in the same scale.

Figure 3.4 illustrates the added advantage of combining graphs. It is clear that the peak in the histogram is almost perfectly aligned with the median shown in the box plot, suggesting the data distribution is nearly symmetric. Additionally, the outliers, hardly visible in the histogram, can easily be spotted in the box plot. This example demonstrates the synergy offered by combining different types of graphs, and the technique used to implement it.

3.2.2 One Categorical Variable

When it comes to categorical variables, we are usually interested in the number of samples fall in each category of a categorical variable. As discussed in Chapter 2, the tallying can be done using the `table()` function (Section 2.1.8) or `group_by()` followed by `summarize(n=n())` (Section 2.2.11).

3.2.2.1 Bar Plot

Bar plot is the standard choice for visualizing tallied data. The only input to the R's `barplot()` graph function is a tabulated count of the categorical variable. Here's an example:

```
par(mfrow=c(2,2))

# Top left
barplot(with(iris, table(Species)))

# Top right
barplot(with(iris, table(Species)),
        xlab="Species",
        ylab="Count",
        main="A Barplot of Species")

# Bottom left
barplot(with(iris, table(Species)),
        col='skyblue',
        border='blue',
        xlab="Species",
        ylab="Count",
        main="Color Options")

# Bottom right
barplot(with(iris, table(Species)),
        horiz = TRUE,
        col='skyblue',
        border='blue',
        xlab="Count",
        ylab="Species",
        main="A Horizontal Barplot")
```



Figure 3.5: A simple bar plot. Top left: Default. Top right: with tables

Sometime you might want to reorder the categories along the x- or y-axis. By default, `barplot()` places the bars from left to right according to the order of the levels. E.g., show the levels of `iris$Species` by:

```
levels(iris$Species)
```

```
[1] "setosa"      "versicolor" "virginica"
```

You can change the order of the levels by updating the factor variable using the `factor()` function (Section 1.2.4). Suppose, the desired order is `virginica`, `setosa`, and `versicolor`. Here's the code:

```
iris$Species <- factor(iris$Species,
                      levels = c("virginica", "setosa", "versicolor"),
                      ordered = TRUE)
levels(iris$Species)
```

```
[1] "virginica"  "setosa"    "versicolor"
```


Here's the revised bar plot.

```
barplot(with(iris, table(Species)),
        col='skyblue',
        border='blue',
        xlab="Species",
        ylab="Count",
        main="Reordered Species")
```

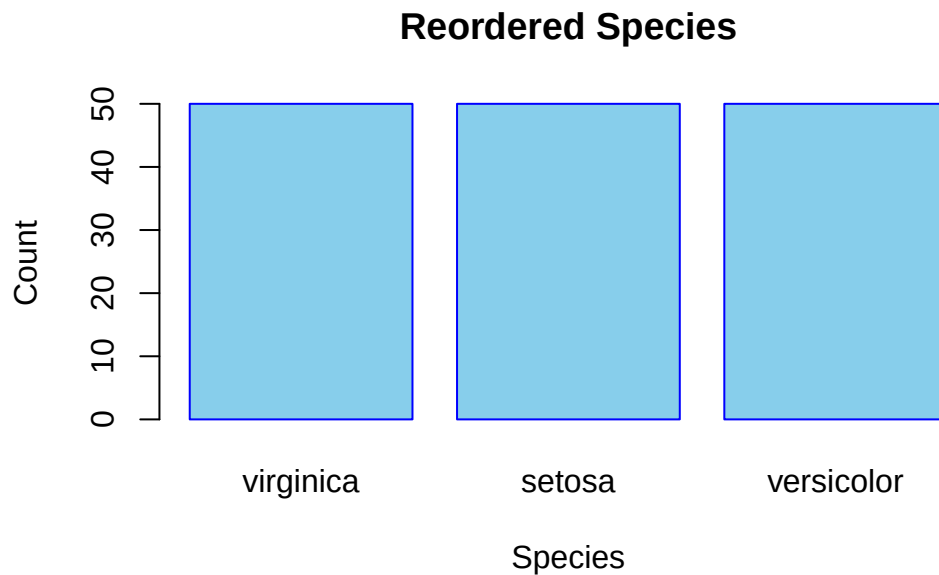


Figure 3.6: Reorder the Bars of a Barplot

3.2.2.2 Pie Chart

An alternative to bar plots is pie charts.

```
par(mfrow=c(2,2))

# Top left
pie(with(iris, table(Species)))

# Top left
pie(with(iris, table(Species)),
```

```

    col = c('skyblue', 'orange', 'white'),
    main="Species")

# Bottom right
counts <- with(iris, table(Species))
my_labs <- paste0(names(counts), "(", counts, ")")
pie(counts,
     labels = my_labs,
     col = c('skyblue', 'orange', 'white'),
     main = "Species Counts")

# Bottom right
counts <- with(iris, table(Species))
fractions <- round(counts/sum(counts), 2)
my_labs <- paste0(names(counts), "(", fractions, ")")
pie(counts,
     labels = my_labs,
     col = c('skyblue', 'orange', 'white'),
     main = "Species Fractions")

```



Figure 3.7: A simple pie chart. Top left: Default. Top right: with tables. Bottom left: Add a title. Show count for each category. Bottom right: Add a fraction or proportion for each category.

3.2.3 One Categorical and One Numeric Variables

The categorical variable in this situation is usually served as a group identifier that splits the data into groups so the numerical variable can be compared between groups. This kind of graph is generally named **side-by-side plot**. You can choose a side-by-side box plot or a side-by-side bar plot. For example, a side-by-side box plot that visually compares the robust statistics of `Sepal.Length` between different species of iris.

3.2.3.1 Side-by-side box plot

```
par(mfrow=c(2,1))

boxplot(Sepal.Length ~ Species,
        data=iris,
        main="A Side-by-Side Box plot")

boxplot(Sepal.Length ~ Species,
```

```
data=iris,
horizontal = TRUE,
main="A Horizontal Side-by-Side Box plot")
```

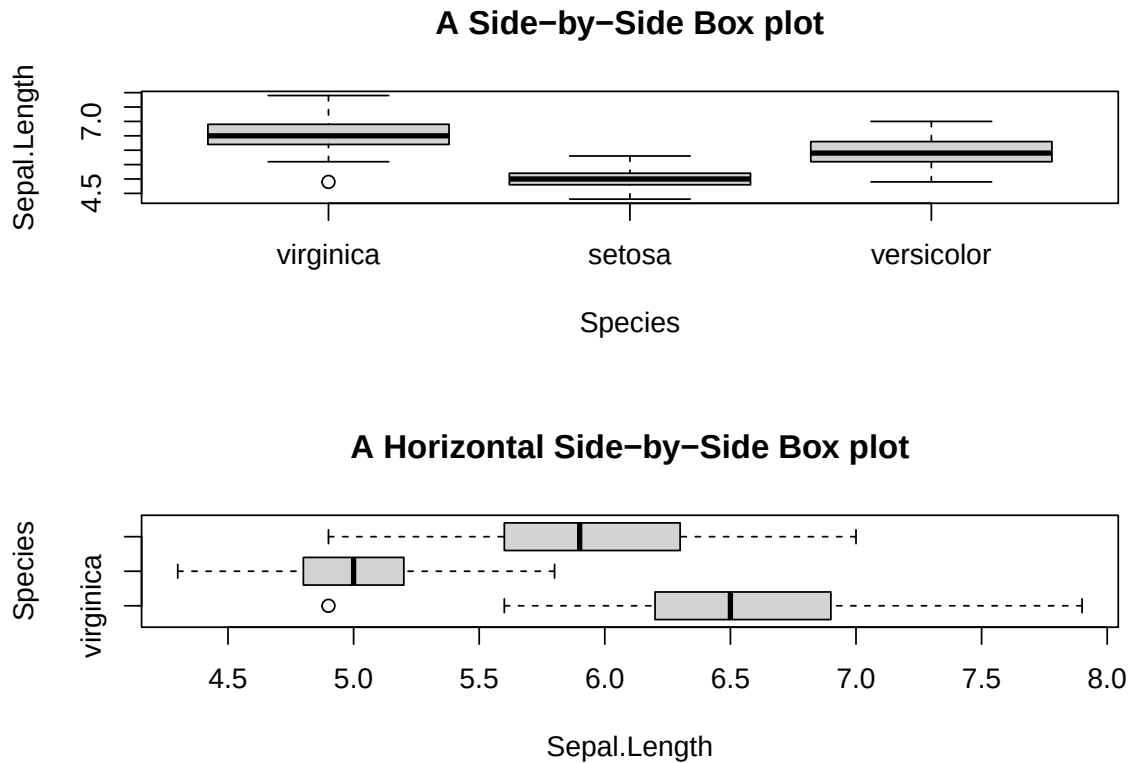


Figure 3.8: A side-by-side box plot

`Sepal.Length ~ Species` is read as “Sepal length **by** species”, the tilde symbol “~” means “by”.

However, there is glitch in the horizontal box plot (bottom): the y-labels collide. It can be fixed by:

1. Expand the left margin of the plotting area.
2. Add the `las=2` parameter to the `boxplot()` function, and disable the default `ylab=`.
3. Manually add the y-label.

```

# Expand the left margin so the text doesn't truncate
# The numbers correspond to margins: c(bottom, left, top, right)
# Default is c(5, 4, 4, 2). We are increasing the left margin from 4 to 7.
par(mar = c(5, 7, 4, 2) + 0.1)

# Draw the plot, but explicitly tell R NOT to draw the y-axis label yet
boxplot(Sepal.Length ~ Species,
        data=iris,
        horizontal = TRUE,
        las = 2,
        ylab = "", # This suppresses the default colliding label
        main="Y-tick Labels Reorientated")

# Add the y-axis label manually, pushing it further away from the axis
# The 'line' argument controls how many text lines away from the axis it sits
title(ylab = "Species", line = 5)

```

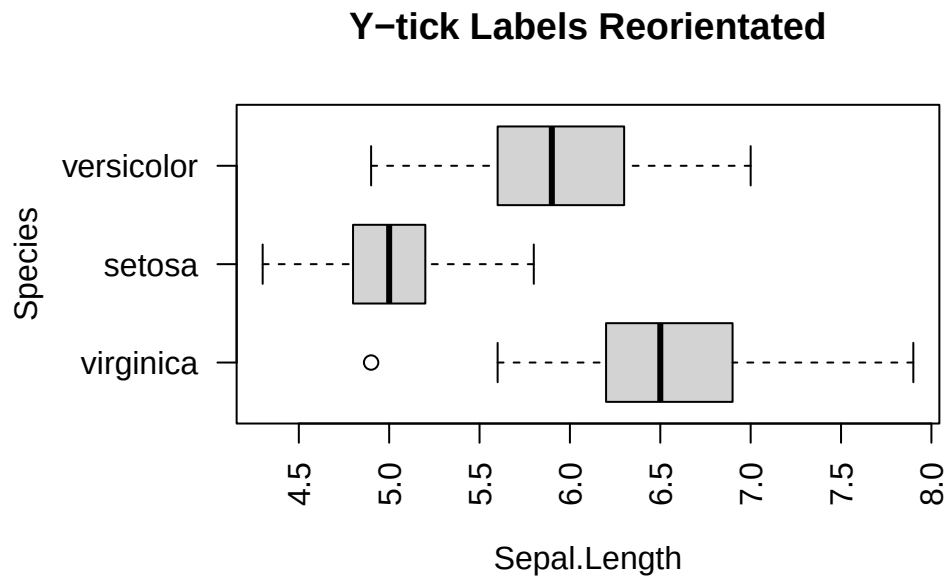


Figure 3.9: A Side-by-Side Box Plot with Horizontal Y-Labels

3.2.3.2 Side-by-side Bar Plot

Suppose you want to visually compare the average sepal length among different iris species. You will begin to feel the cumbersome of base R, and appreciate the power of **tidyverse**. Anyway, you will see how to make such a graph with a new function, **tapply()**, in two steps.

First, make a table of average sepal length by species. Apparently, the **table()** function may help. But it does not work for this case, as it only tallies the number of samples by species. Hence, you need to use a new function **tapply()**. Here's an example:

```
mean_data <- tapply(iris$Sepal.Length, iris$Species, FUN = mean)
mean_data
```

virginica	setosa	versicolor
6.588	5.006	5.936

How does **tapply()** work? The “t” represents table. The function applies the **mean** function to the input data **iris\$Sepal.Length** grouped by **iris\$Species**. In other words, the **mean()** is applied to the sepal lengths for each species separately.

And then, pass the **mean_data** object to **barplot()**.

```
barplot(mean_data,
        xlab="Species",
        ylab="Average Sepal.Length",
        main="Side-by-side Barplot")
```

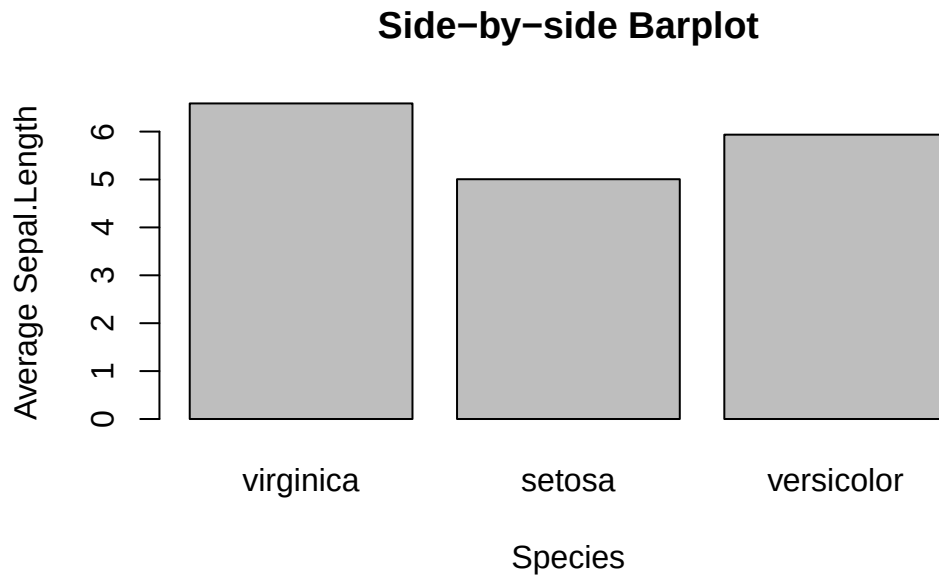


Figure 3.10: A Side-by-Side Bar Plot

3.2.3.3 Side-by-side Bar Plot with Error Bar^{*}

This topic is slightly advanced. It is much easier to use `ggplot2`. So, feel free to skip this or jump to Section 3.3.4.3.

The error bar represents the standard error of the mean (SEM). Here is the formula:

$$\bar{x} \pm \frac{s}{\sqrt{n}}$$

, where s and n are the standard deviation and sample size, respectively. In the bar plot you are going to plot, the height of the bar is $\bar{x}$, and the error bars span above and below the bar's top by $\frac{s}{\sqrt{n}}$ units.

First, calculate the mean and standard of errors using the `tapply()` function.

```
mean_data <- tapply(iris$Sepal.Length, iris$Species, FUN = mean)
mean_data
```

```
virginica    setosa versicolor
  6.588      5.006      5.936
```

```
sem <- function(x) {sd(x)/sqrt(length(x))}
sem_data <- tapply(iris$Sepal.Length, iris$Species, FUN = sem)
sem_data
```

```
virginica      setosa versicolor
0.08992695 0.04984957 0.07299762
```

First, create the bar plot and store the height of bars in the object `bar_midpoints`. Use the `arrows()` function to create/add the error bars to the bar plot, where each error bar consists of the upper whisker (x_1, y_1) and lower whisker (x_0, y_0) .

```
bar_midpoints <- barplot(mean_data,
                          xlab="Species",
                          ylab="Average Sepal.Length",
                          main="Side-by-side Barplot")

arrows(x0 = bar_midpoints,
       y0 = mean_data - sem_data, # Start of the bar
       x1 = bar_midpoints,
       y1 = mean_data + sem_data, # End of the bar
       angle = 90,                # Makes the ends of the bars flat
       code = 3,                  # Draws ticks at both ends
       length = 0.5, col='red')  # Sets the length of the ticks
```

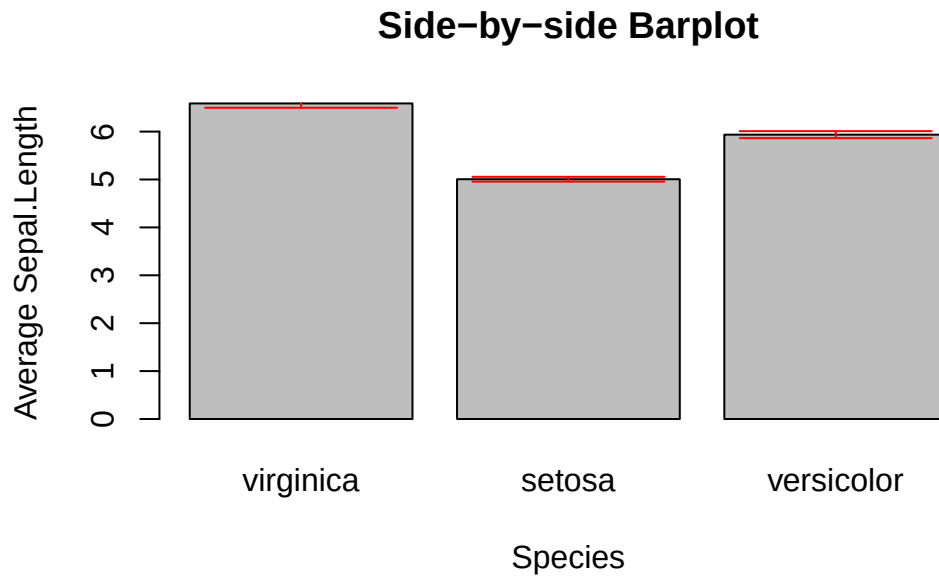



Figure 3.11: A Side-by-side Bar Plot with Error Bars

Once again, if you found this confusing, see the `ggplot` version of error bars (Section [3.3.4.3](#)).

3.2.3.4 Line Graph with Error Bar

If your message is to show a trend from left to right, a line graph is the default choice. Here is an example to create a line graph with solid dots.

```
# Default line plot
plot(mean_data,
     type='b',
     pch=19,
     xlab="Species",
     ylab="Average Sepal.Length",
     main="A Line Plot")
```

A Line Plot

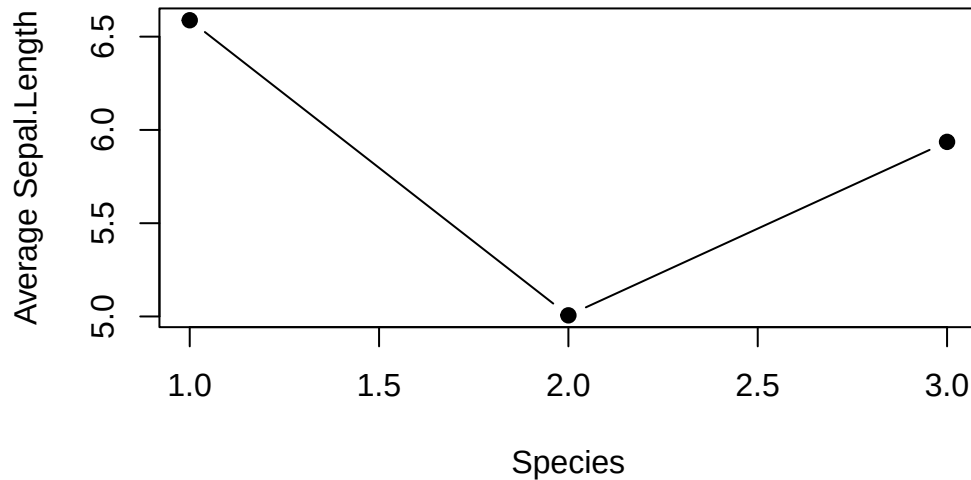


Figure 3.12: A Line Plot

- Use `pch=19` to specify the shape of the data points. `pch` stands for **p**lotting **c**haracter. The value 19 denotes a solid dot (●).
- To join the dots together in a line plot, specify `type='b'`, the **b** means **both** dots and line are plotted. More options of `type` can be found using the help function: `help(plot)` documentation.

But, there is a glitch in the x-tick. The x-tick should show the actual species names instead of number. It can be fixed in two steps: 1. Suppress the default x-tick labeling feature of the `plot()` function by setting the parameter `xaxt="n"`. And then, paste the customized x-tick back by the `axis()` function. The species names can be obtained by `names(mean_data)`.

```
# Fix x-tick labels
plot(mean_data,
     type='b',
     pch=19,
     xlab="Species",
     ylab="Average Sepal.Length",
     main="Change x-tick Labels",
     yaxt="n")
axis(side=1, at=1:length(mean_data), labels=names(mean_data))
```

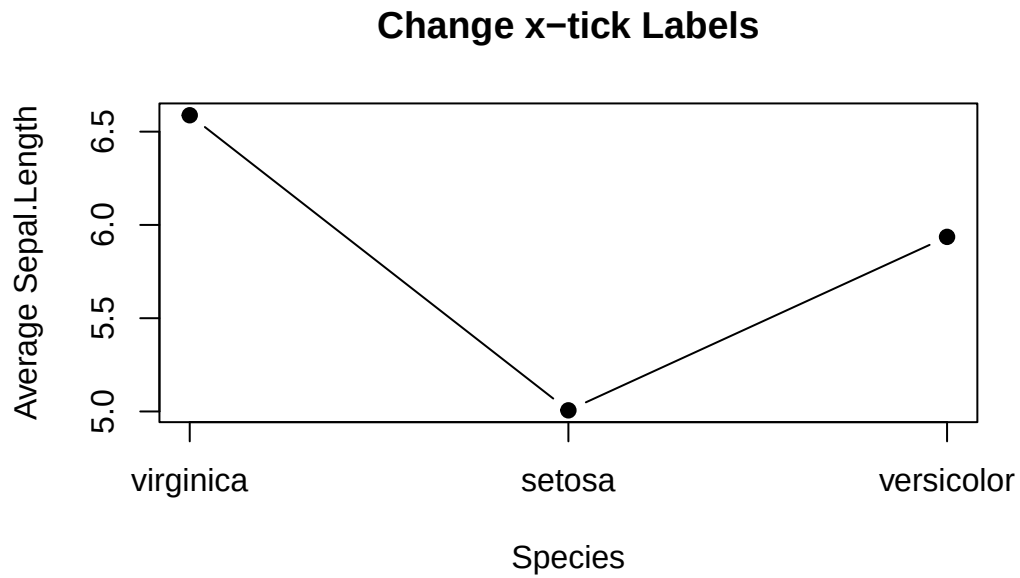


Figure 3.13: A Line plot with proper x-tick labels

Now we can add error bars to the line plot. It is similar to the bar plot above, except that `x0` and `x1` should take the values 1,2,3.

```
# Add Error Bar
plot(mean_data,
     type='b',
     pch=19,
     xlab="Species",
     ylab="Average Sepal.Length",
     main="Change x-tick Labels",
     xaxt="n")
axis(side=1, at=1:length(mean_data), labels=names(mean_data))

arrows(x0 = seq(length(mean_data)),
      y0 = mean_data - sem_data, # Start of the bar
      x1 = seq(length(mean_data)),
      y1 = mean_data + sem_data, # End of the bar
      angle = 90,                # Makes the ends of the bars flat
      code = 3,                  # Draws ticks at both ends
      length = 0.05, col='red') # Sets the length of the ticks
```

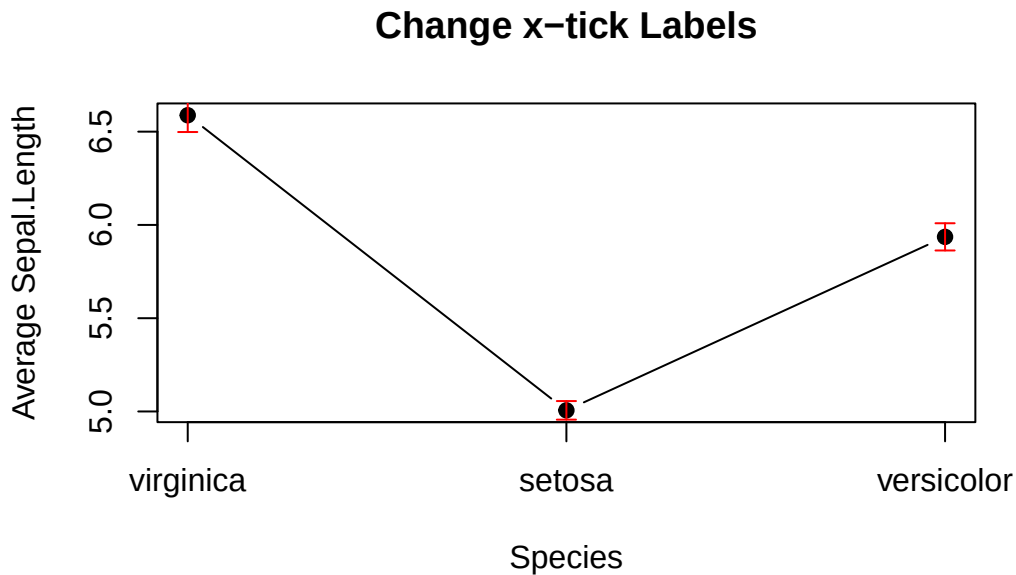


Figure 3.14: A Line plot with Error Bars

3.2.3.5 Overlapping Histogram

The code for creating this plot can be convoluted. It is easier to make it using `ggplot2`. Feel free to skip to Section 3.3.4.5.

Suppose you are interested in visualizing and comparing the distribution of sepal lengths among the three iris species. Here are the steps to do it:

1. Split the data by individual species.
2. Calculate the range for the x- and y-axis so as to ensure the histograms are in the same scale.
3. Create a histogram in certain color for a species from step 1, and use it as a template.
4. Overlay the histograms from other species on top of each other.

Let's do it step by step below:

```
# Step 1: Split the data by individual species  
setosa <- subset(iris, Species == "setosa")
```

```
versicolor <- subset(iris, Species == "versicolor")
virginica <- subset(iris, Species == "virginica")
```

```
# Step 2: Set up the range for the x- and y-axis
```

```
x_range <- range(iris$Sepal.Length)
y_range <- c(0, 25) # 25 is arbitrary.
```

Below is the code for plotting the template histogram, and how additional histograms are added to it. To distinguish species, different color is attributed to different species. It is done by `col = rgb(...)`. The `rgb()` function stands for red, green, and blue. It takes four parameters: the intensity of red, green, blue, and transparency. All are in the range of 0 and 1. E.g., `rgb(1, 0, 0, 0.5)` means 100% red, no green and blue at all, and transparency is 50%.

```
# Step 3: Create a template
```

```
hist(setosa$Sepal.Length,
     main = "Overlapping Histogram of Sepal Length",
     xlab = "Sepal Length",
     col = rgb(1, 0, 0, 0.5), # Red with 50% transparency
     breaks = seq(4,9,by=0.25),
     xlim = x_range,
     ylim = y_range)
```

```
# Step 4: Overlay histograms from other species
```

```
hist(versicolor$Sepal.Length, col=rgb(0, 1, 0, 0.5), add = TRUE) # green with 50% transparency
hist(virginica$Sepal.Length, col=rgb(0, 0, 1, 0.5), add = TRUE) # blue with 50% transparency
```

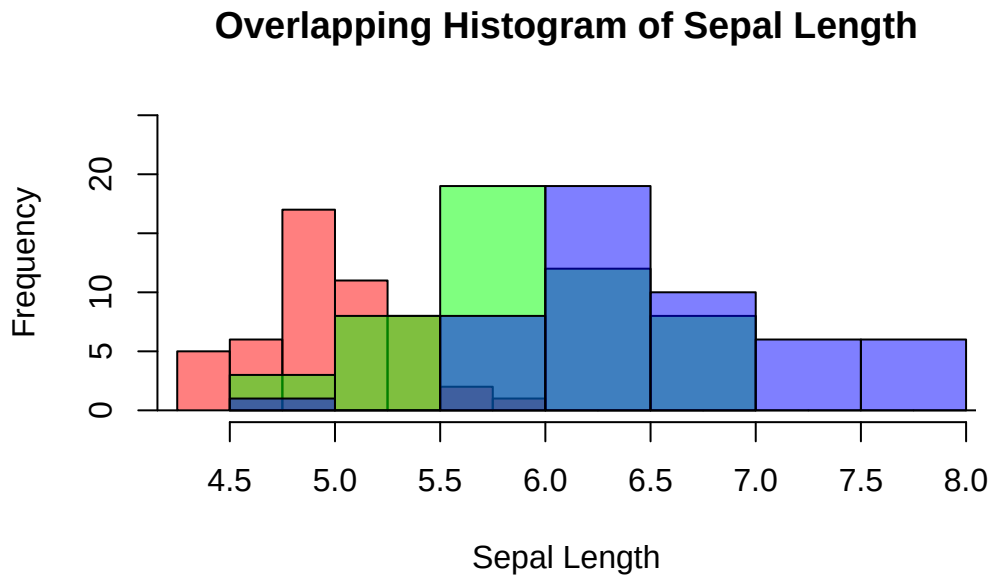


Figure 3.15: Overlapping histograms

3.2.4 Contingency Table

Here we deal with a data set that comprises two categorical variables. A two-dimensional contingency table is used to tally the number of samples for different combinations of categories of the two categorical variables.

The distribution of the counts can reveal whether or not the two categorical variables are dependent. To illustrate how to visualize a contingency table, we will use the built-in data set `penguins`. The data set contains two categorical variables: `species` and `island`.

```
summary(penguins)
```

species	island	bill_len	bill_dep
Adelie :152	Biscoe :168	Min. :32.10	Min. :13.10
Chinstrap: 68	Dream :124	1st Qu.:39.23	1st Qu.:15.60
Gentoo :124	Torgersen: 52	Median :44.45	Median :17.30
		Mean :43.92	Mean :17.15
		3rd Qu.:48.50	3rd Qu.:18.70
		Max. :59.60	Max. :21.50
		NAs :2	NAs :2

flipper_len	body_mass	sex	year
Min. :172.0	Min. :2700	female:165	Min. :2007
1st Qu.:190.0	1st Qu.:3550	male :168	1st Qu.:2007
Median :197.0	Median :4050	NAs : 11	Median :2008
Mean :200.9	Mean :4202		Mean :2008
3rd Qu.:213.0	3rd Qu.:4750		3rd Qu.:2009
Max. :231.0	Max. :6300		Max. :2009
NAs :2	NAs :2		

Suppose you are interested to explore whether or not different species has shown preference in habitat or geographical location (`island`).

Let's begin with creating a 2D table as below:

```
mytab <- with(penguins, table(species, island))
mytab
```

	island		
species	Biscoe	Dream	Torgersen
Adelie	44	56	52
Chinstrap	0	68	0
Gentoo	124	0	0

Obviously, `species` and `island` are not independent, as you can see not all three species are present in all islands. If `species` and `island` are independent, the counts in the contingency table should be below:

```
indep_tab
```

	island		
species	Biscoe	Dream	Torgersen
Adelie	74	55	23
Chinstrap	33	25	10
Gentoo	61	45	19

Two popular graphs are suitable for visualizing a contingency table: a side-by-side bar plot, or a mosaic plot.

```
par(mfrow=c(3,1))

barplot(mytabs,
        beside=T,
        xlab = "Island",
        ylab = "Count",
        main = "A 2D Table",
        col = c("skyblue", "lightgreen", "lightcoral"),
        legend.text = rownames(mytabs))

mosaicplot(mytabs, main = "Mosaic Plot of Two Dependent Variables")

mosaicplot(indep_tabs, main = "Mosaic Plot of Two Independent Variables")
```

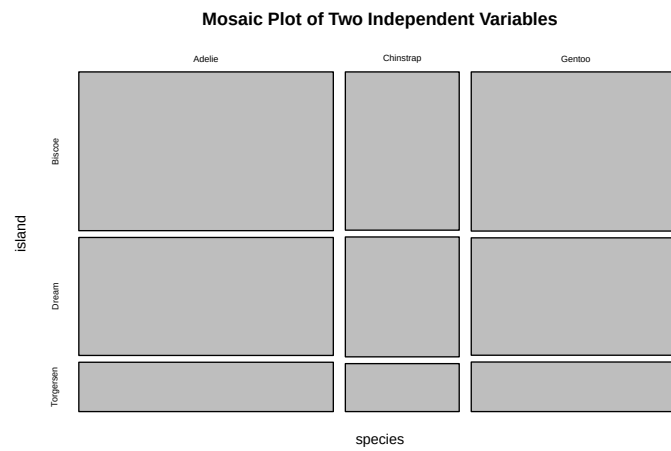
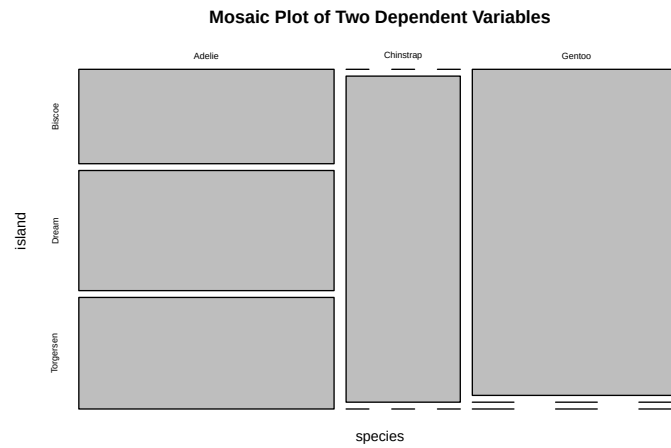
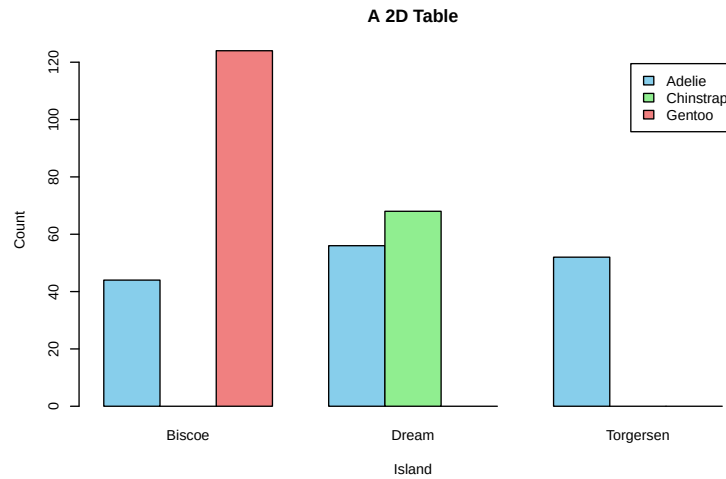



Figure 3.16: Visualization of a 2D Contingency Table. Top. A side-by-side barplot. Middle. Mosaic plot. Bottom. Mosaic plot for independent categorical variables

3.2.5 Two Numeric Variables

A scatter plot is the default choice to visualize the relationship between two numeric variables. The relationship can be manifold, ranging from linear to cyclical. So, visualizing how the data points are distributed in a 2D plan is an intuitive way to learn about their relationship. Importantly, visualization should be done before performing any statistical analysis, e.g., linear regression. Here is how to create a scatter plot that shows the relationship between sepal width and sepal length. Note that each point (x_i, y_i) is a pair of values from one sample.

```
par(mfrow=c(1,2))

# Top left
plot(x=iris$Sepal.Width,
     y=iris$Sepal.Length,
     xlab = "Sepal Width",
     ylab = "Sepal Length",
     main = "Default")

# Top right
plot(x=iris$Sepal.Width,
     y=iris$Sepal.Length,
     pch=3,
     xlab = "Sepal Width",
     ylab = "Sepal Length",
     main = "Change the Point Style")
```

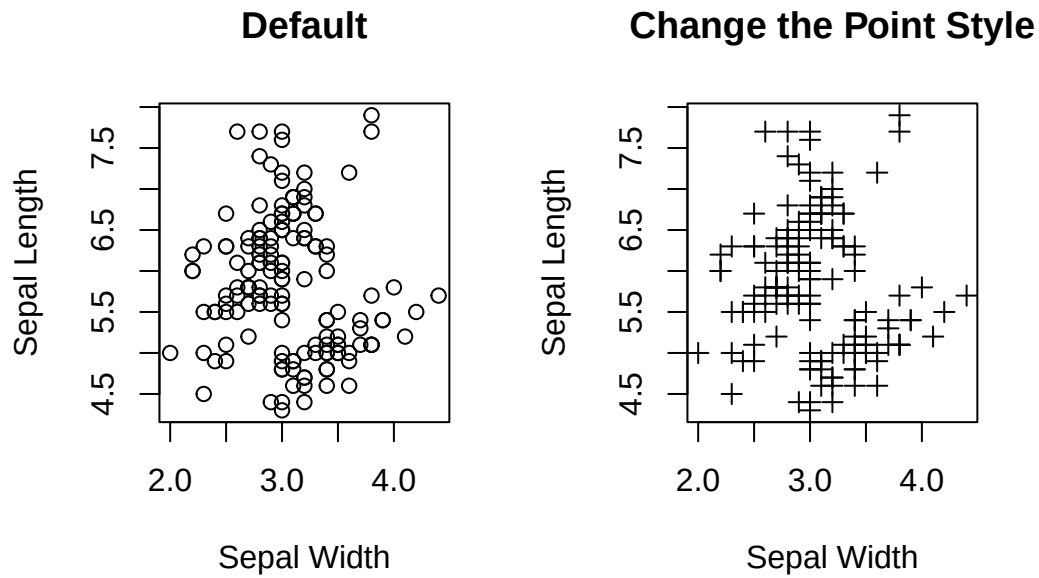


Figure 3.17: Visualizing the relationship between two numeric variables.

```
plot(x=iris$Sepal.Width,
     y=iris$Sepal.Length,
     pch=3,
     xlab = "Sepal Width",
     ylab = "Sepal Length",
     main = "Segregate Data Points by Color (Species)",
     col = c("skyblue", "lightgreen", "lightcoral")[as.numeric(iris$Species)])

legend("topright",
      legend = levels(iris$Species),
      col = c("skyblue", "lightgreen", "lightcoral"),
      pch = 3,
      title = "Species")
```

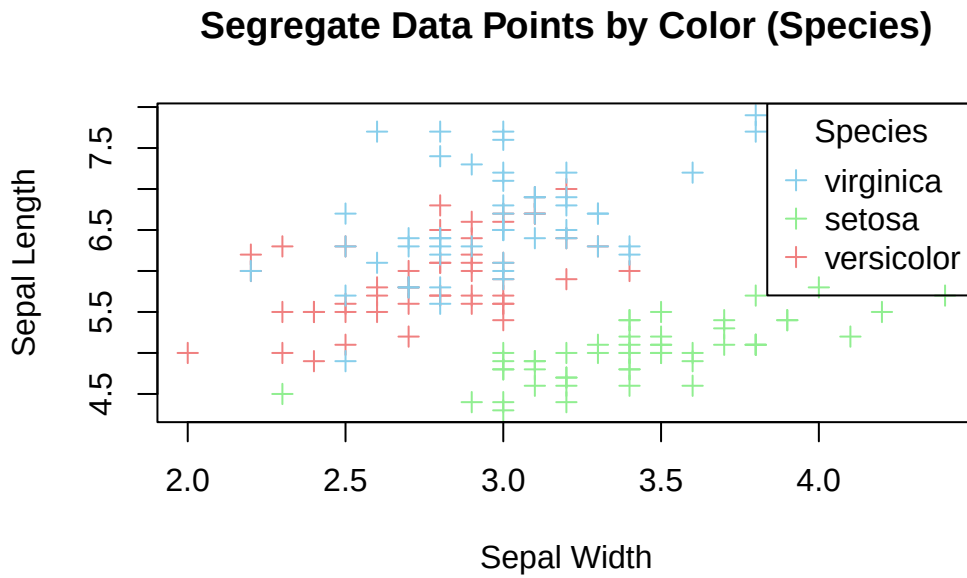


Figure 3.18: Color data points by species.

If your aim is to explore linear relationship between the two variables, it can be emphasized by adding a regression line for the two variables on top of the data points. Here is an example, regress `Sepal.Length` (response) on `Sepal.Width` (independent).

```
lm_model <- lm(Sepal.Length ~ Sepal.Width, data=iris)

# intercept
intercept <- lm_model$coefficients[1]

# slope
slope = lm_model$coefficients[2]

print(paste("Slope =", slope, "Intercept =", intercept))
```

```
[1] "Slope = -0.2233610611299 Intercept = 6.52622255089448"
```

```
plot(x=iris$Sepal.Width,
     y=iris$Sepal.Length,
     pch=3,
     xlab = "Sepal Width",
```

```

ylab = "Sepal Length",
main = "Add a Regression Line",
col = c("skyblue", "lightgreen", "lightcoral")[as.numeric(iris$Species)]

legend("topright",
      legend = levels(iris$Species),
      col = c("skyblue", "lightgreen", "lightcoral"),
      pch = 3,
      title = "Species")

# Add a line
abline(a=intercept, b=slope, col='red')

```

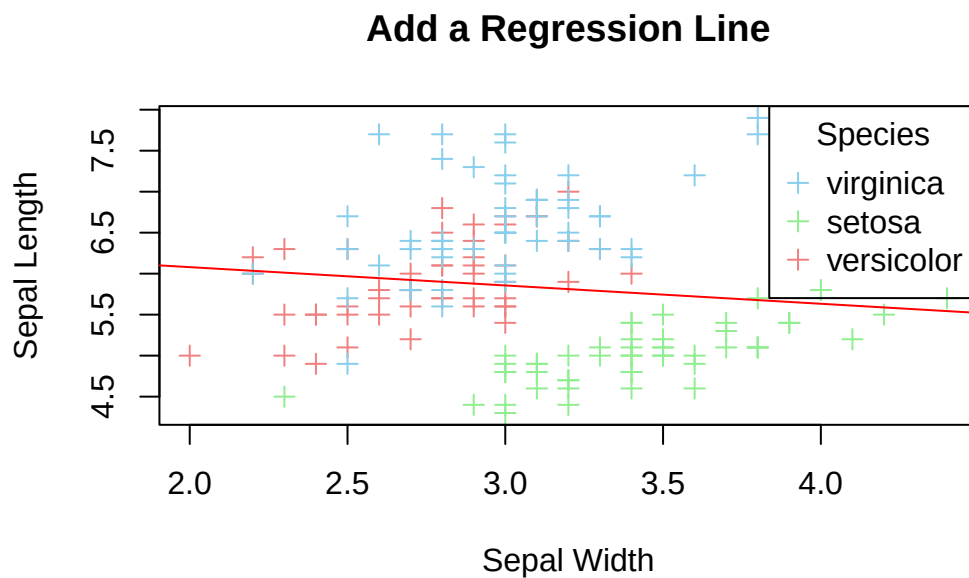


Figure 3.19: A scatter plot with a regression line.

3.2.6 Multipanel Scatter Plot

A multipanel scatter plot offers a glimpse of the relationship between all possible pairs of numeric variables. Suppose, you are interested in the pairwise relationship between all numeric variables in `iris`. It can easily be done using the `pairs()` plot function.

```
pairs(iris[,1:4], main="A Multi-Panel Scatter Plot")
```

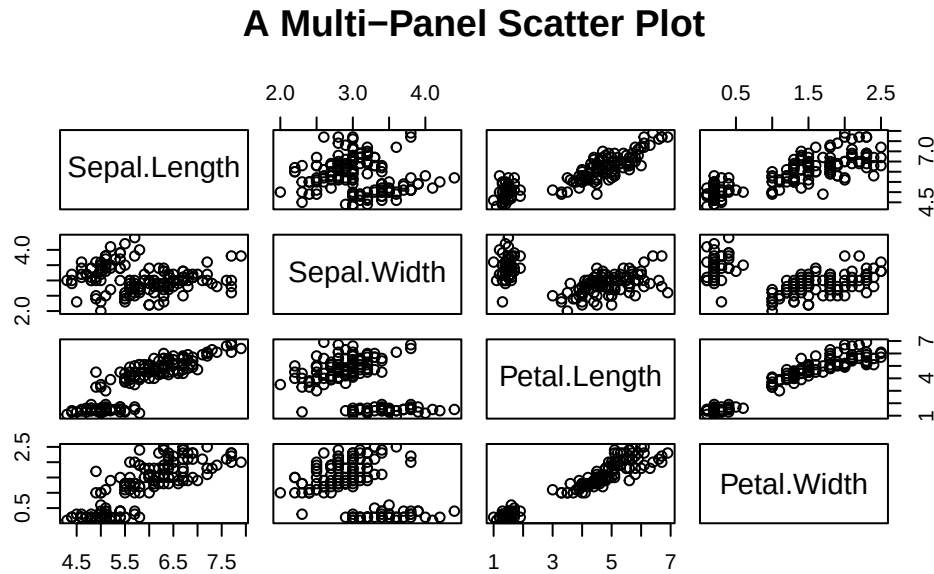


Figure 3.20: A Multipanel Scatter Plot

Here it concludes our journey of data visualization using base R functions. As you can see, some plots can be made easily using base R (e.g., multipanel scatter plot), while others are less direct (e.g., bar plot with error bars). There is no one system you should use, choose whatever fits your requirements.

3.3 ggplot2

`ggplot2` is bundled in `tidyverse`. Once you have activated `tidyverse`, `ggplot2` is available. Here, the focus is on using `ggplot2` to plot the graphs that were created using base R above. We will not repeat the discussion of strengths and weaknesses of each graph in this section. Readers should refer to the corresponding section above for the characteristics and functions of the graphs.

If `tidyverse` has not been activated, load it into R before using `ggplot2`.

```
require(tidyverse)
```

Loading required package: tidyverse

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.2
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

3.3.1 Grammar of ggplot2

The two 'g's of `ggplot2` stand for **Grammar of Graphics**. In `ggplot2`, the graphical widgets are arranged in layers. That idea gives developers the flexibility to decorate a graph by adding layers to an existing graph. The **data** layer forms the foundation of a graph. The **Aesthetics** defines the association between data variable(s) and the axes. **Geometries** are the types of graphs, e.g., line plot, histogram, etc. **Facets** split a graph into multi-panels. **Statistics** defines how to aggregate data, e.g., in proportion or take the face value in a barplot. **Coordinates** supports an X-Y Cartesian coordinate system, and a polar coordinate system. (See the Pie Chart section below) **Themes** offers different color schemes, font type, font size, etc.

3.3.2 One Numeric Variable

You can plot a histogram of the numeric variable to see its distribution, such as the spread, the peak, symmetry, and flatness. If your concern is robust statistics, make a box plot instead.

3.3.2.1 Histogram

Let's redraw the histogram in Figure 3.1 using `ggplot2`. The first line is the *data layer* that specifies the data source, and it must be a data frame (or tibble), e.g., `iris`. The *aesthetic* option designates the x-axis for `Sepal.Width`. The second line is the *geometry* of the graph. For a histogram, the function name is `geom_histogram()`. Here is the bare minimum of a `ggplot2` histogram:

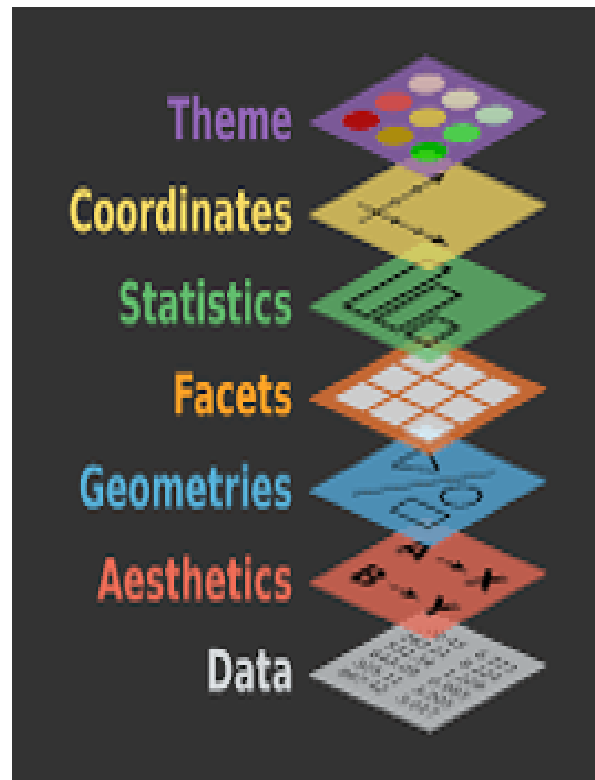


Figure 3.21: Anatomy of ggplot2


```
# Default
ggplot(data=iris, aes(x=Sepal.Width)) +
  geom_histogram()
```

``stat_bin()` using `bins = 30`. Pick better value `binwidth`.`

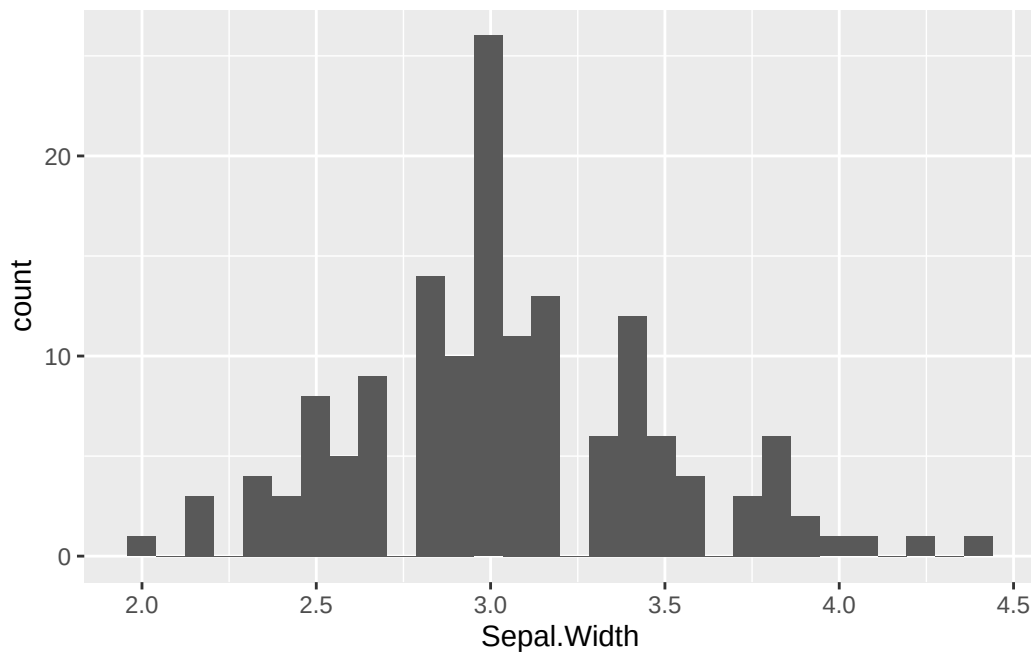


Figure 3.22: ggplot2 Histogram.

If you see this message: “`stat_bin()` using `bins = 30`. Pick better value `binwidth`”, R just informs you that 30 bins was used by default. You can ignore it or specify a new `binwidth` value.

It is always a good idea to give the graph a title at the top so people know what the key message is. Equally important is the x- and y-labels. This task can be done by the `labs()` function. By default, the title is left-justified. To center it, see the last line and the comment above it.

```
# Overwrite default x-/y-label and title
ggplot(data=iris, aes(x=Sepal.Width)) +
  geom_histogram() +
  labs(x="Sepal Width", y="Count", title="Histogram of Sepal Width") +
```

```
# hjust = 0.5 means the horizontal adjustment is the midpoint.
theme(plot.title = element_text(hjust = 0.5))
```

`stat\_bin()` using `bins = 30`. Pick better value `binwidth`.

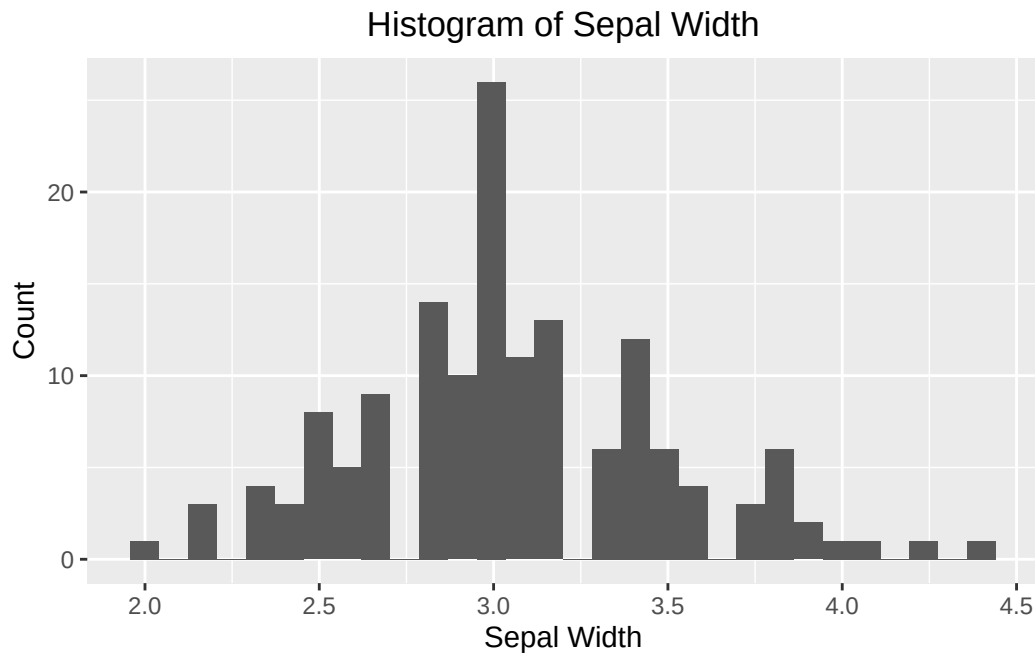


Figure 3.23: A ggplot2 Histogram with customized labels and title.

Change the fill color and border color. It can be done by providing additional parameters to the `geom_histogram()`.

```
# fill = filled color
# color = border color
ggplot(data=iris, aes(x=Sepal.Width)) +
  geom_histogram(fill="skyblue", color = "white") +
  labs(x="Sepal Width", y="Count", title="Color Options") +
  # hjust = 0.5 means the horizontal adjustment is the midpoint.
  theme(plot.title = element_text(hjust = 0.5))
```

`stat\_bin()` using `bins = 30`. Pick better value `binwidth`.

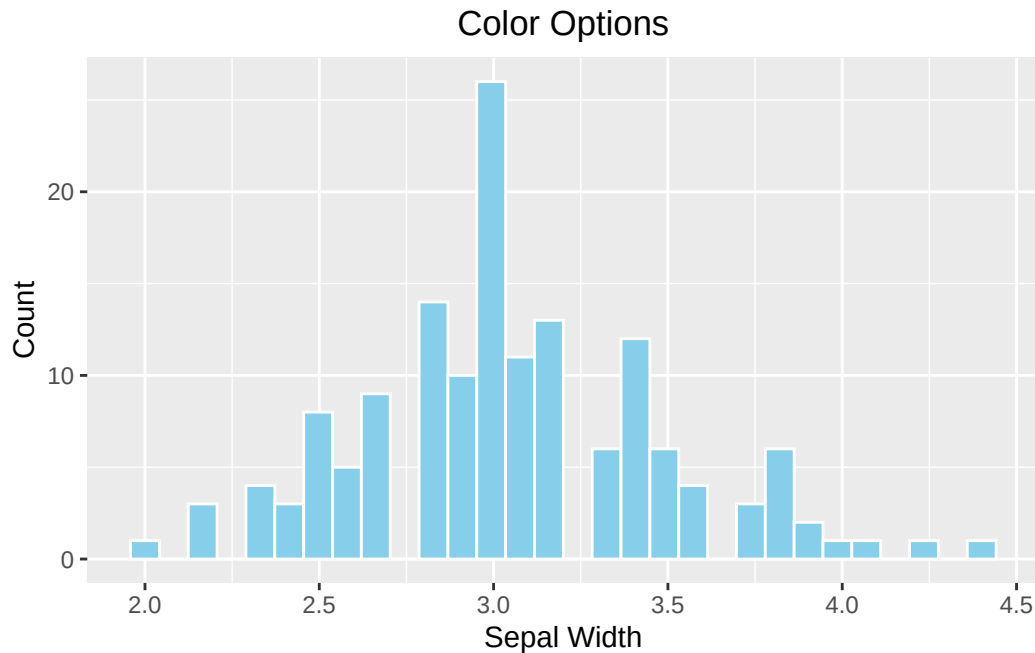


Figure 3.24: A `ggplot2` Histogram with a customized color.

The bin width of a histogram is calculated automatically by `ggplot2` according to the range of `x`. You can use the `breaks` parameter to overwrite it.

```
# breaks
ggplot(data=iris, aes(x=Sepal.Width)) +
  geom_histogram(fill="skyblue", color = "white", breaks=seq(2,4.5,by=0.1)) +
  labs(x="Sepal Width", y="Count", title="`breaks` option") +
  # hjust = 0.5 means the horizontal adjustment is the midpoint.
  theme(plot.title = element_text(hjust = 0.5))
```

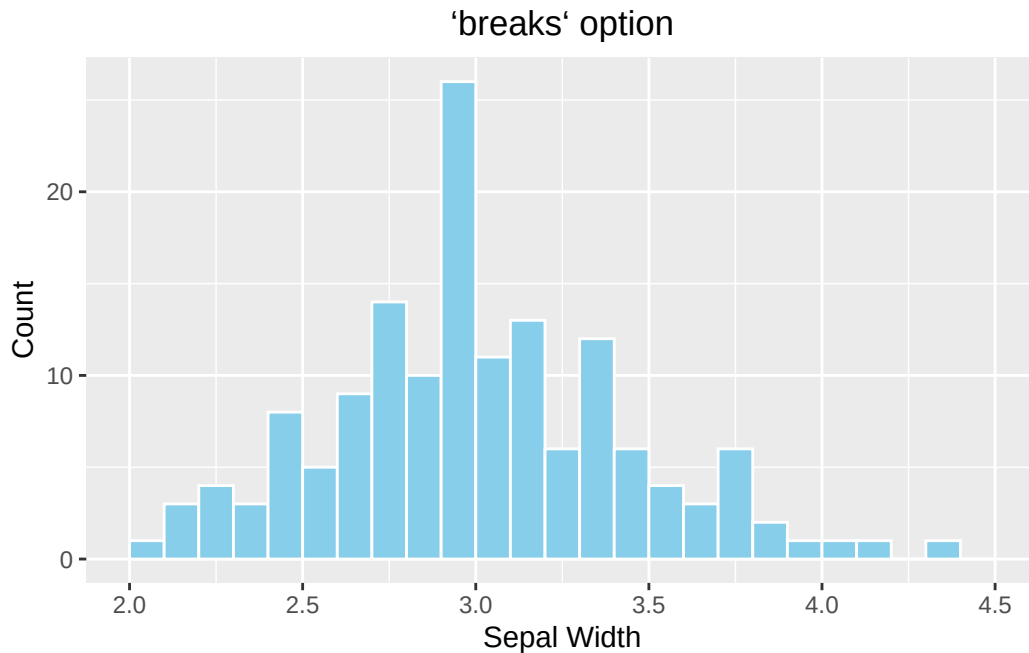


Figure 3.25: A `ggplot2` Histogram with a non-default bin width.

Optional topic

As you might notice that each of the four histograms occupies the entire plotting area instead of displaying in a 2x2 grid as Figure 3.1. It is because `ggplot2` does not work with `par(mfrow(2,2))`. To divide the plotting area into a grid that is compatible with `ggplot2`, use the `patchwork` package. Use the menu option **Tools** → Install Packages to install it.

Another new concept is that a graph can be stored in a variable so it can be displayed later in the grid. In the example below, the four histograms above are initially stored in variables `p1`, `p2`, `p3`, and `p4`. See the example below:

```
require(patchwork)

# Default
p1 <- ggplot(data=iris, aes(x=Sepal.Width)) +
  geom_histogram()

# With labels and title
p2 <- ggplot(data=iris, aes(x=Sepal.Width)) +
  geom_histogram() +
  labs(x="Sepal Width", y="Count", title="Histogram of Sepal Width") +
  # hjust = 0.5 means the horizontal adjustment is the midpoint.
```

```

    theme(plot.title = element_text(hjust = 0.5))

# fill = filled color
# colour = border color
p3 <- ggplot(data=iris, aes(x=Sepal.Width)) +
  geom_histogram(fill="skyblue", color = "white") +
  labs(x="Sepal Width", y="Count", title="Color Options") +
  # hjust = 0.5 means the horizontal adjustment is the midpoint.
  theme(plot.title = element_text(hjust = 0.5))

# Change the number of bins
p4 <- ggplot(data=iris, aes(x=Sepal.Width)) +
  geom_histogram(fill="skyblue", color = "white", breaks=seq(2,4.5,by=0.1)) +
  labs(x="Sepal Width", y="Count", title="`breaks` option") +
  # hjust = 0.5 means the horizontal adjustment is the midpoint.
  theme(plot.title = element_text(hjust = 0.5))

# Arrange them in a 2x2 grid
# '+' places plots next to each other, '/' stacks them
(p1 + p2) / (p3 + p4)

```

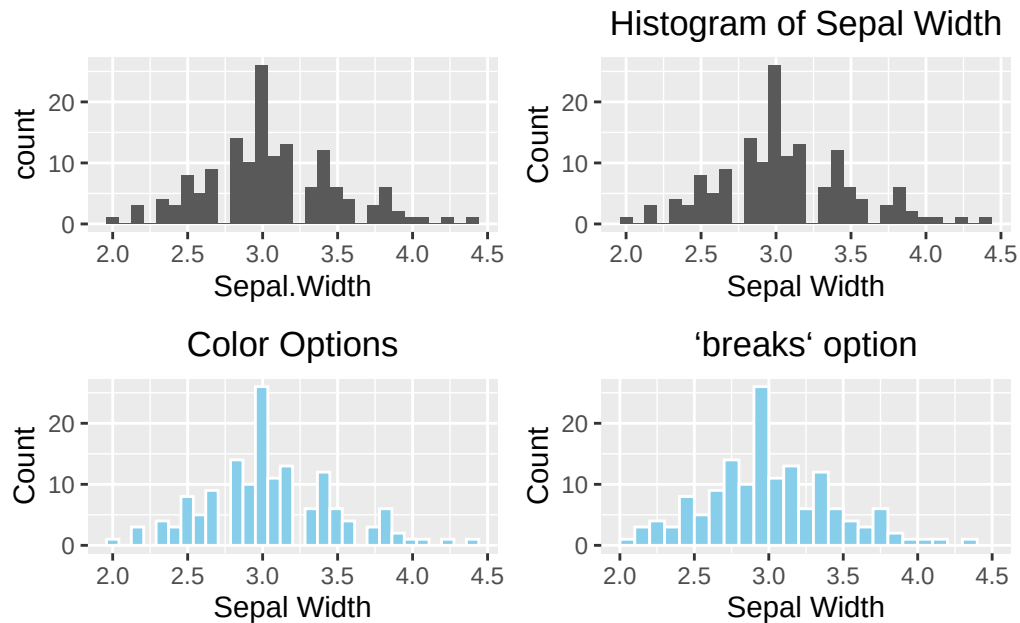


Figure 3.26: ggplot2 Grid System by `patchwork`. Top left: Default (bare minimum). Top right: with labels and title. Bottom left: change the color. Bottom right: change the number of bins.

3.3.2.2 Box plot

The anatomy of a box plot can be found in Figure 3.3. The geometry function of a box plot is `geom_boxplot()`. Unlike a histogram, the numeric variable is mapped to the y-axis for a vertical box plot. If you want a horizontal box plot, map the numeric variable to the x-axis, and don't forget to specify the label for the x-axis.

```
require(patchwork)

# Default
p1 <- ggplot(data=iris, aes(y=Sepal.Width)) +
  geom_boxplot()

# Add y-label and title
p2 <- ggplot(data=iris, aes(y=Sepal.Width)) +
  geom_boxplot() +
  labs(y="Sepal Width", title = "box plot of Sepal Width") +
  theme(plot.title = element_text(hjust = 0.5))
```

```
# Change color
p3 <- ggplot(data=iris, aes(y=Sepal.Width)) +
  geom_boxplot(fill="skyblue", outlier.color = "red") +
  labs(y="Sepal Width", title = "Color options") +
  theme(plot.title = element_text(hjust = 0.5))

# Horizontal box plot
p4 <- ggplot(data=iris, aes(x=Sepal.Width)) +
  geom_boxplot() +
  labs(x="Sepal Width", title = "Horizontal box plot") +
  theme(plot.title = element_text(hjust = 0.5))

(p1 + p2)/(p3 + p4)
```

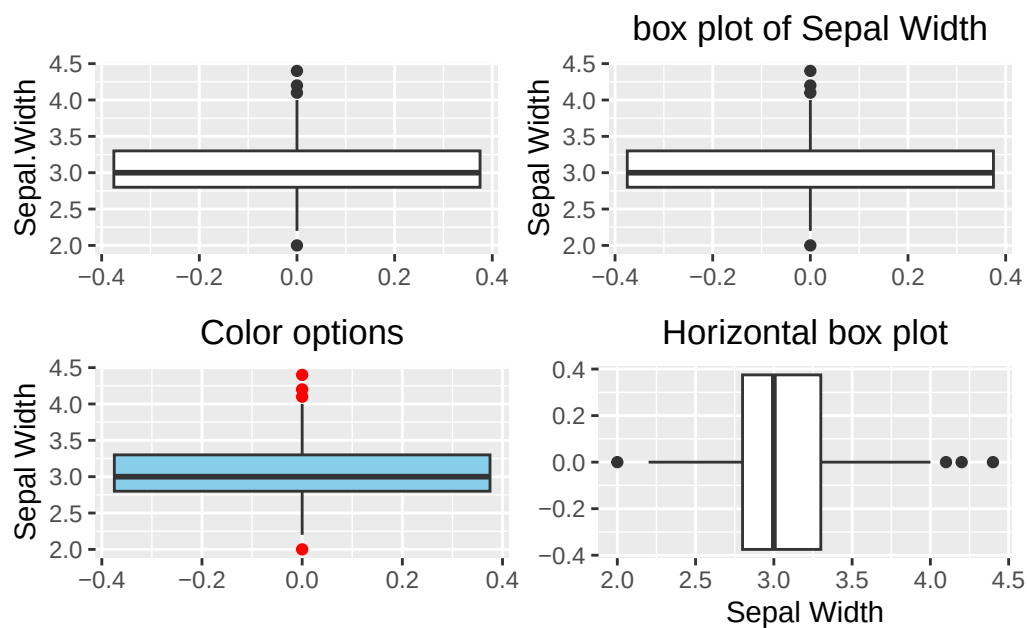


Figure 3.27: box plot using `ggplot2`. Top left: default (bare minimum). Top right: with y-label and title. Bottom left: change color. Bottom right: horizontal box plot.

3.3.3 One Categorical Variable

3.3.3.1 Bar Plot

Here we will show how to make a bar plot and a pie chart using `ggplot2`.

```
require(patchwork)

# Default
p1 <- ggplot(data=iris, aes(x=Species)) +
  geom_bar()

# With labels and title
p2 <- ggplot(data=iris, aes(x=Species)) +
  geom_bar() +
  labs(x="Species", y="Count", title = "A Barplot of Species") +
  theme(plot.title = element_text(hjust = 0.5))

# Color option
p3 <- ggplot(data=iris, aes(x=Species)) +
  geom_bar(fill="skyblue", color="blue") +
  labs(x="Species", y="Count", title = "A Barplot of Species") +
  theme(plot.title = element_text(hjust = 0.5))

# Horizontal barplot
p4 <- ggplot(data=iris, aes(y=Species)) +
  geom_bar(fill="skyblue", color="blue") +
  labs(y="Species", x="Count", title = "A Horizontal Barplot of Species") +
  theme(plot.title = element_text(hjust = 0.5))

(p1 + p2)/(p3 + p4)
```

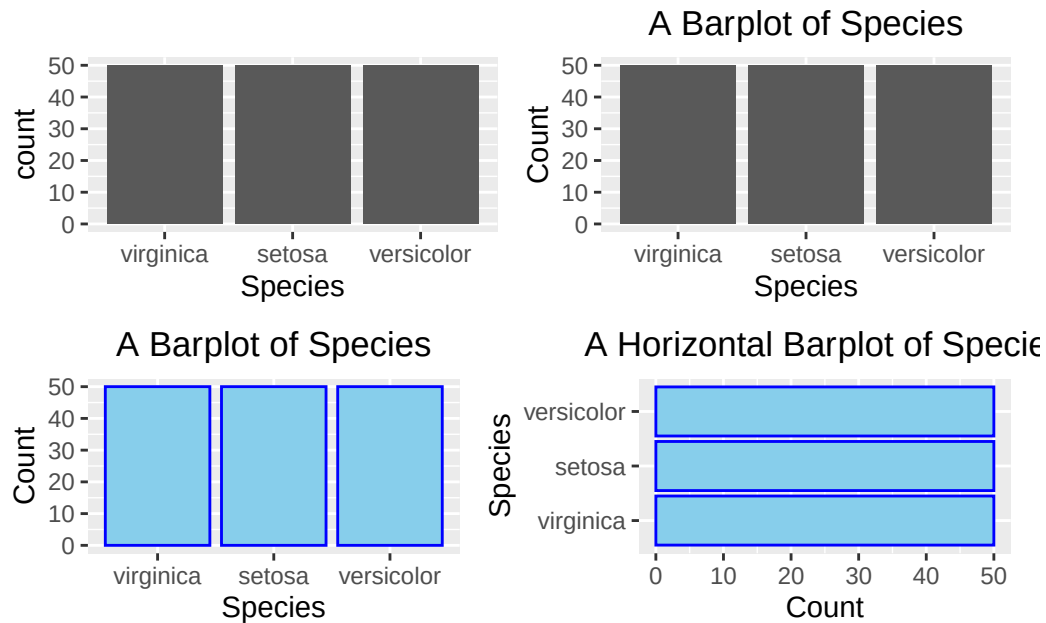



Figure 3.28: Barplot using `ggplot2`. Top left: default (bare minimum).

3.3.3.2 Pie Chart

Here's a curve ball. You may give an impression that `ggplots` is an upgrade. Plotting a pie chart should be a piece of cake. The fact is NO.

Despite the the disappointment, it is worth to see how a pie chart can be plotted using the foundational building-block functions. To give you a heads-up, `ggplot2` views a pie chart as a twisted bar plot. There is an internal logic to justify such a view but we will not discuss it in detail.

Suppose we want to create a pie chart like the one below:

Here we will walk through each of the intermediate steps until getting the final pie chart.

Let's begin with a stacked barplot, where the bars are stacked up into a single column, and the fill-color is an attribute to distinguish different iris species (`fill=Species`). The stacked bar is placed at the position 2 units on the right from the origin (`aes(x=2)`), but the exact position does not matter.

```
ggplot(data=iris, aes(x=2, fill=Species)) +  
  geom_bar()
```

Donut Chart of Iris Species

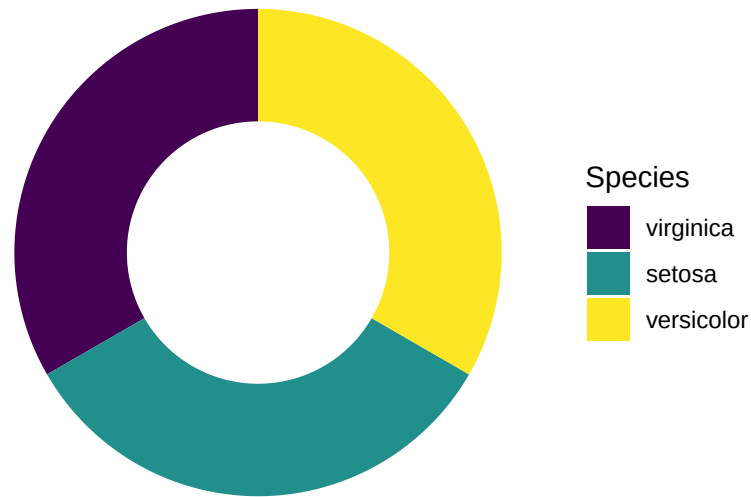


Figure 3.29: Pie chart or a donut chart

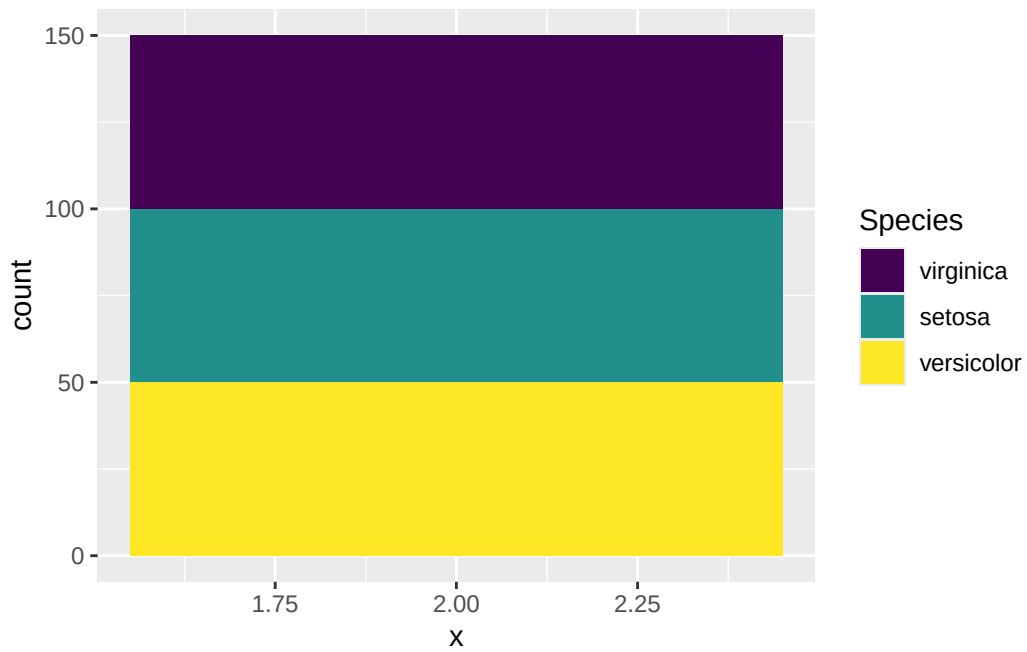


Figure 3.30: A stacked barplot

Now, use your imagination to picture that the stacked bar is bent into a ring. Does it look like the pie chart in Figure 3.30? To way to “bend” it in `ggplot2` is to change the x-y Cartesian coordinate system to the polar (circular) coordinate system in which a point is characterized by two parameters: the distance from the origin (r), and the angle (θ) from the horizontal **polar axis** in an counter-clockwise direction.

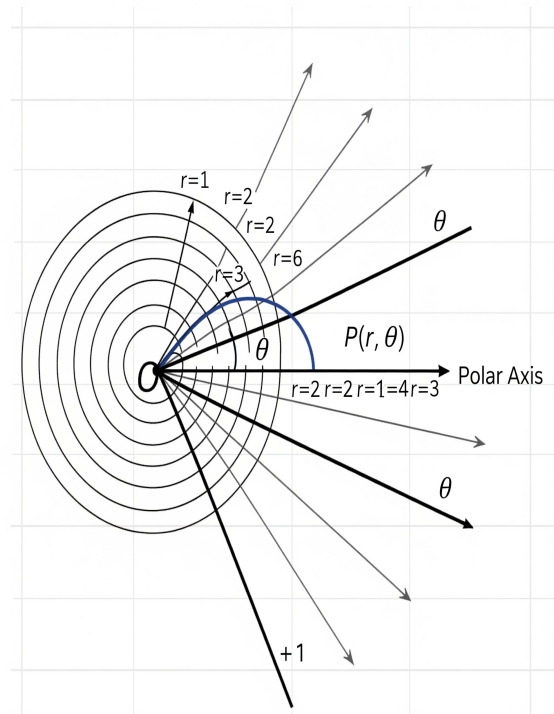


Figure 3.31: Polar Coordinate. (generated by Gemini)

By adding this line `coord_polar(theta="y", start = 0)` in the code below, it transforms the height of the bar for each species into an angle (θ) in the polar coordinate, where the zero position is the 12 o'clock (`start = 0`). In this example, one third of the samples is `setosa`. As such, the angle θ is $360^\circ \times \frac{1}{3} = 120^\circ$. If the proportion is a quarter, the angle $\theta = 360^\circ \times \frac{1}{4} = 90^\circ$.

```
ggplot(data=iris, aes(x="", fill=Species)) +
  geom_bar() +
  coord_polar(theta="y", start = 0)
```

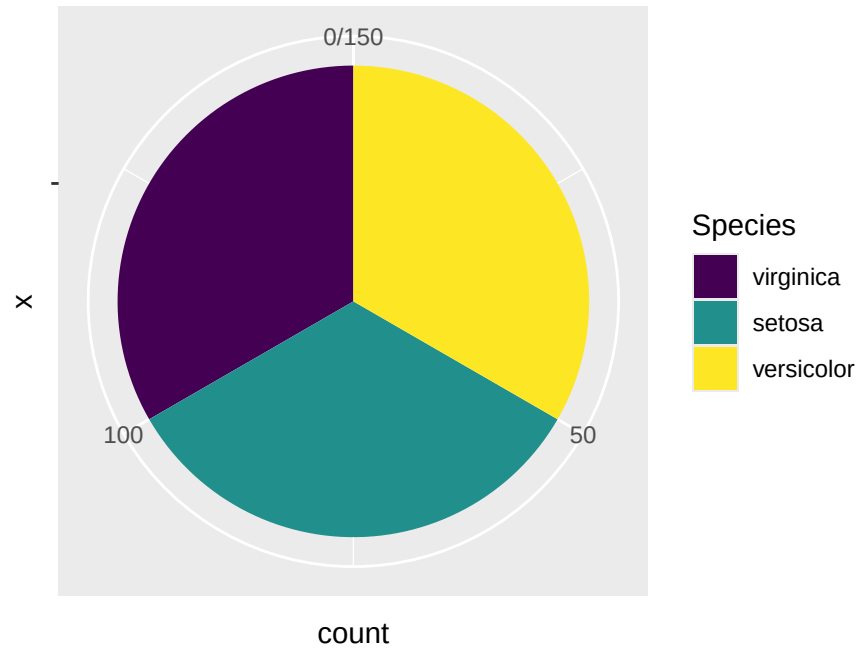


Figure 3.32: A Piechart by changing to the polar coordinate system.

The final step is to open a hole in the pie chart if you like, for aesthetic reasons. But this modification is optional.

```
ggplot(iris, aes(x = 2, fill = Species)) +
  geom_bar(width = 1, stat = "count") +
  coord_polar(theta = "y", start = 0) +
  # This is the key part that creates the hole
  xlim(0.5, 2.5) +
  labs(title = "Donut Chart of Iris Species")
```

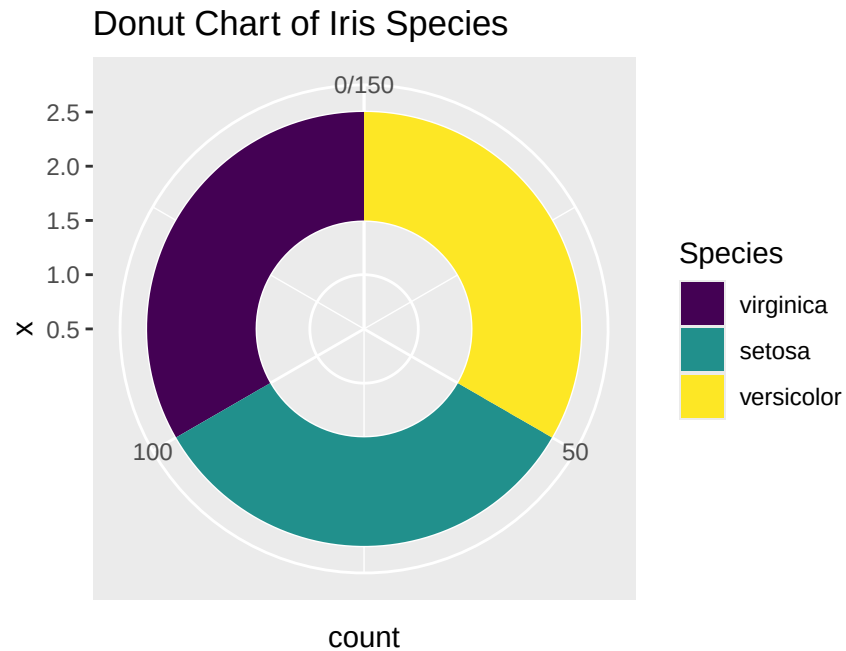


Figure 3.33: A Piechart using `ggplot2`.

If you prefer to have a clean background, add the line `theme_void()`. Finally, here's the desired pie chart. What a journey!

```
ggplot(iris, aes(x = 2, fill = Species)) +
  geom_bar(width = 1, stat = "count") +
  coord_polar(theta = "y", start = 0) +
  # This is the key part that creates the hole
  xlim(0.5, 2.5) +
  theme_void() +
  labs(title = "Donut Chart of Iris Species")
```

Donut Chart of Iris Species

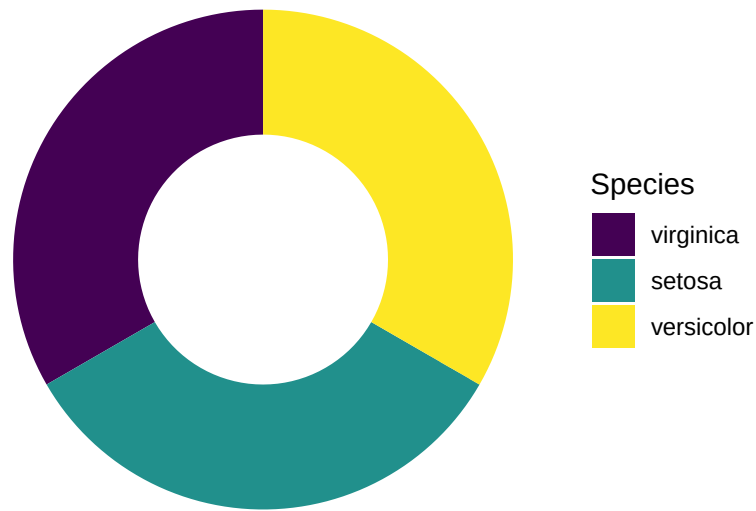


Figure 3.34: Piechart using ggplot2 without no background.

3.3.4 One Categorical and One Numeric Variables

3.3.4.1 Side-by-side Box Plot

Here you want to compare the robust statistics among different species.

```
require(patchwork)

p1 <- ggplot(data=iris, aes(x=Species, y=Sepal.Length)) +
  geom_boxplot() +
  labs(title = "A Side-by-Side box plot") +
  theme(plot.title = element_text(hjust = 0.5))

p2 <- ggplot(data=iris, aes(x=Sepal.Length, y=Species)) +
  geom_boxplot() +
  labs(title = "A Horizontal Side-by-Side box plot") +
  theme(plot.title = element_text(hjust = 0.5))

p1/p2
```

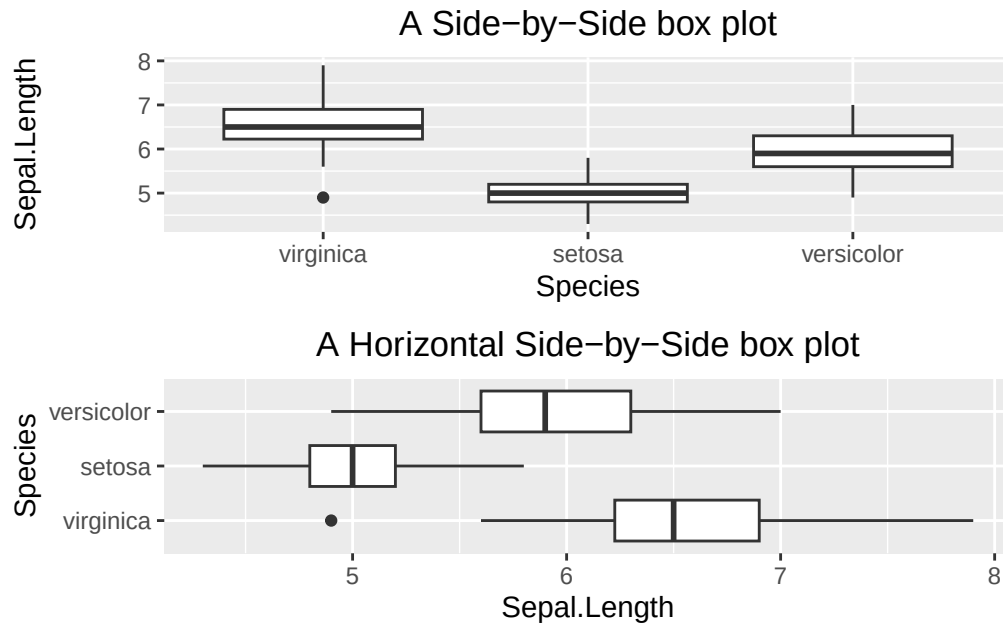


Figure 3.35: A side-by-side box plot

3.3.4.2 Side-by-side Bar Plot

Here, you want to compare the average `Sepal.Length` among iris species. As discussed above, you calculate the averages and store them in a separate table. It makes use of pipe, group by, and summarize concepts discussed in Section 2.2.11.

```
iris_summary1 <- iris %>%
  group_by(Species) %>%
  summarize(mean_sepal_length = mean(Sepal.Length))
iris_summary1
```

```
# A tibble: 3 x 2
  Species    mean_sepal_length
  <ord>          <dbl>
1 virginica      6.59
2 setosa         5.01
3 versicolor     5.94
```

And then, you can plot the summary table. The geometry function is `geom_bar(stat = "identity")`. The `stat = "identity"` parameter tells the `geom_bar()` function to set up the height of the bars according to the face value of `mean_sepal_length`.

```
ggplot(data=iris_summary1, aes(x=Species, y=mean_sepal_length)) +  
  geom_bar(stat = "identity") +  
  labs(x="Species", y="Average Sepal Length", title = "Side-by-side Barplot") +  
  theme(plot.title = element_text(hjust = 0.5))
```

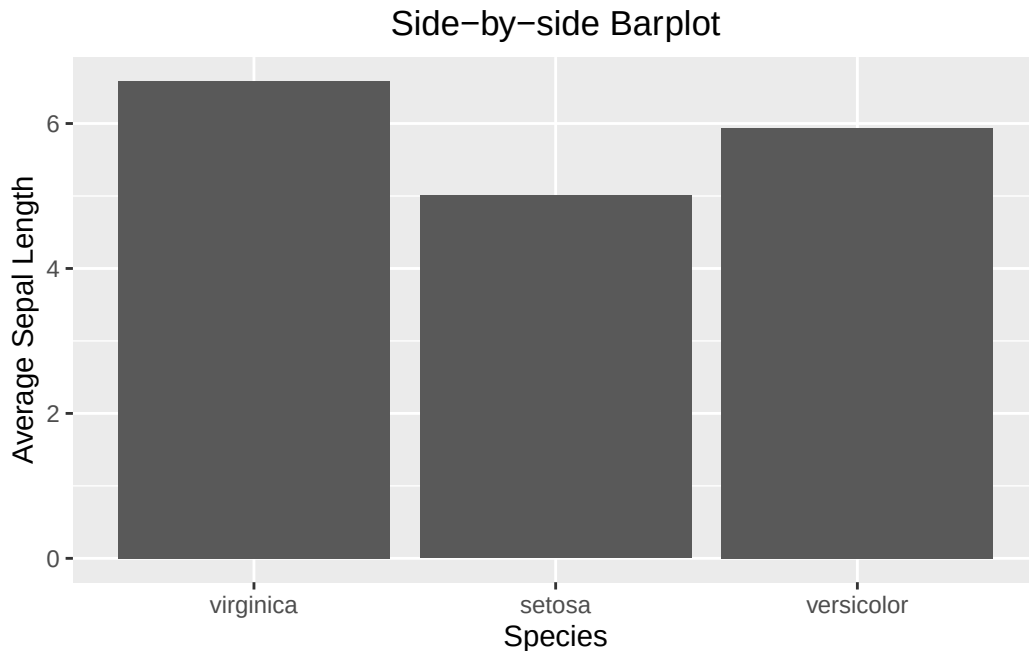


Figure 3.36: A side-by-side bar plot

3.3.4.3 Side-by-side Bar Plot with Error Bar

The calculation of the error bar (i.e., the standard error `se`) requires standard deviation and sample size. Once `se` is calculated, it can be used to calculate the minimum (lower) and maximum (upper) of the error bar.

```
iris_summary2 <- iris %>%  
  group_by(Species) %>%  
  summarize(mean_sepal_length = mean(Sepal.Length),  
            N=n(),  
            se=sd(Sepal.Length)/sqrt(N)) %>%
```



```
# add the y minimum and y maximum columns
mutate(ymin = mean_sepal_length - se,
       ymax = mean_sepal_length + se)

iris_summary2
```

```
# A tibble: 3 x 6
```

	Species	mean_sepal_length	N	se	ymin	ymax
	<ord>	<dbl>	<int>	<dbl>	<dbl>	<dbl>
1	virginica	6.59	50	0.0899	6.50	6.68
2	setosa	5.01	50	0.0498	4.96	5.06
3	versicolor	5.94	50	0.0730	5.86	6.01

```
ggplot(data=iris_summary2, aes(x=Species, y=mean_sepal_length)) +
  geom_bar(stat = "identity") +
  geom_errorbar(aes(ymin = ymin, ymax = ymax), width = 0.2, color='red') +
  labs(x="Species", y="Average Sepal Length", title = "Side-by-side Barplot") +
  theme(plot.title = element_text(hjust = 0.5))
```

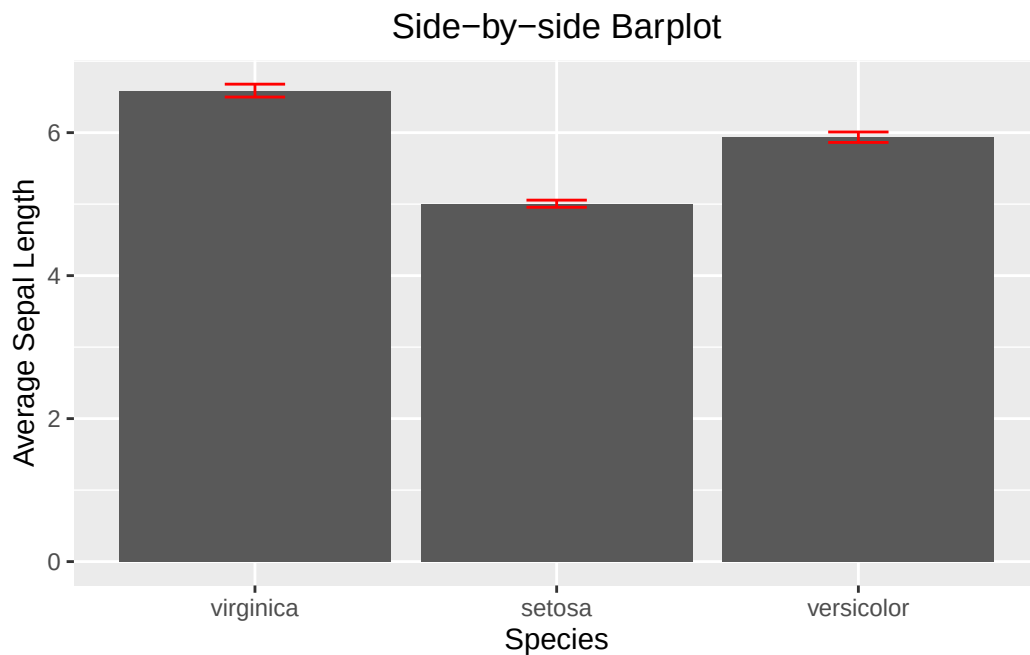


Figure 3.37: A side-by-side bar plot with error bar.

3.3.4.4 Line Graph with Error Bar

Calculate the SE as above:

```
iris_summary3 <- iris %>%
  group_by(Species) %>%
  summarize(mean_sepal_length = mean(Sepal.Length),
            N=n(),
            se=sd(Sepal.Length)/sqrt(N)) %>%
  mutate(ymin=mean_sepal_length-se,
         ymax=mean_sepal_length+se)
iris_summary3
```

```
# A tibble: 3 x 6
  Species    mean_sepal_length      N      se  ymin  ymax
  <ord>          <dbl> <int>   <dbl> <dbl> <dbl>
1 virginica         6.59    50 0.0899  6.50  6.68
2 setosa            5.01    50 0.0498  4.96  5.06
3 versicolor        5.94    50 0.0730  5.86  6.01
```

Incorporate the SE into a line plot.

```
ggplot(iris_summary3, aes(x=Species, y=mean_sepal_length, group = 1)) +
  geom_line(color = 'skyblue') +
  geom_point(size = 3, color = 'skyblue') +
  geom_errorbar(aes(ymin = ymin, ymax = ymax), width = 0.1, linewidth = 1) +
  labs(x="Species", y="Sepal Length", title = "Line Plot with Errorr Bar") +
  theme(plot.title = element_text(hjust = 0.5))
```

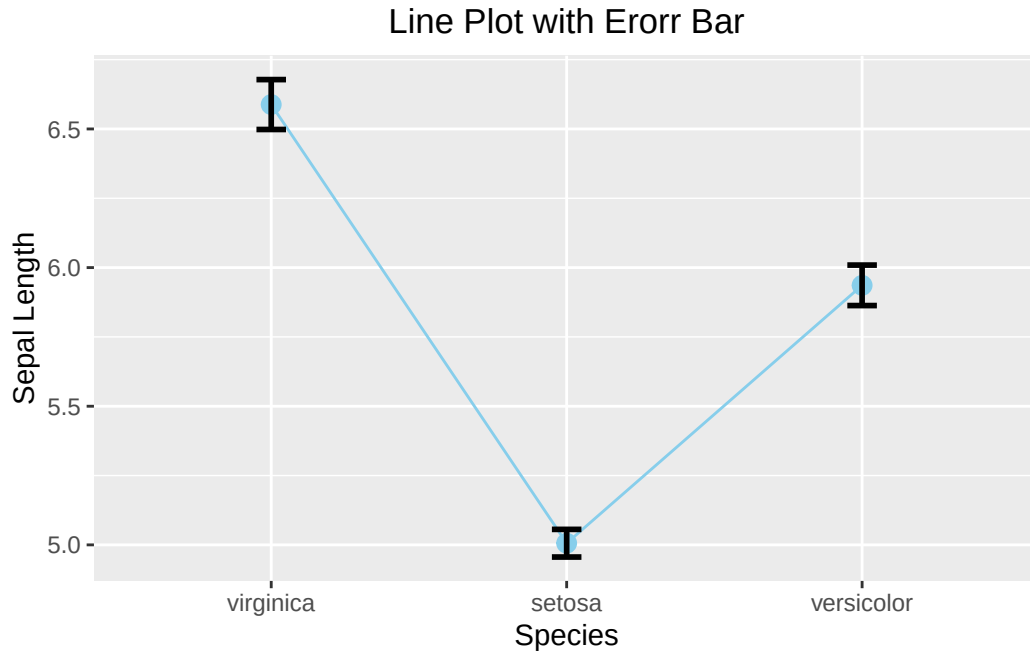


Figure 3.38: A line plot with error bar.

3.3.4.5 Overlapping Histogram

Like the base R side-by-side histogram, you will set the transparency to 0.5 and bin width to 0.25. Note that there is no need to care about the x- and y-limit in `ggplot2`.

```
ggplot(data=iris, aes(x=Sepal.Length, fill=Species)) +  
  geom_histogram(alpha=0.5, binwidth = 0.25) +  
  labs(x="Sepal Length", title = "Overlapping Histogram") +  
  theme(plot.title = element_text(hjust = 0.5))
```

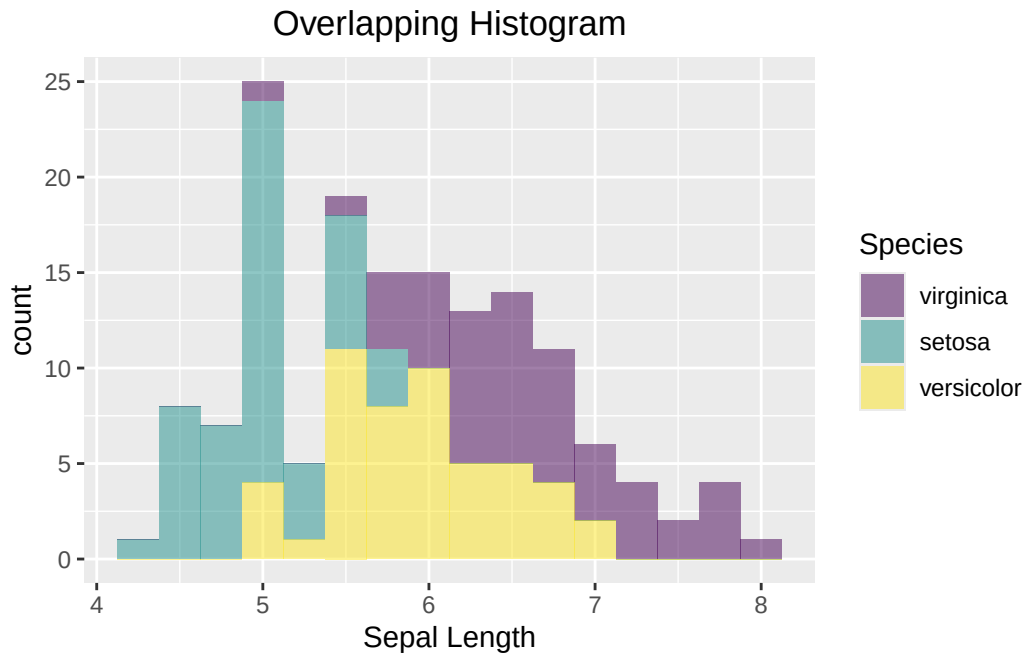


Figure 3.39: A side-by-side histogram.

3.3.5 Contingency Table

The `penguins` data set is used to illustrate how to create a side-by-side bar plot or a mosaic plot for displaying count data.

First, let's create a contingency table as below:

```
mytab <- with(penguins, table(species, island))
mytab
```

	island		
species	Biscoe	Dream	Torgersen
Adelie	44	56	52
Chinstrap	0	68	0
Gentoo	124	0	0

As `ggplot2` only works with data frames, `mytab` is **not** a data frame. Therefore, the first step is to convert the contingency table into a data frame.

```
mydf <- as.data.frame(mytab)
colnames(mydf) <- c("species", "island", "Count")
mydf
```

	species	island	Count
1	Adelie	Biscoe	44
2	Chinstrap	Biscoe	0
3	Gentoo	Biscoe	124
4	Adelie	Dream	56
5	Chinstrap	Dream	68
6	Gentoo	Dream	0
7	Adelie	Torgersen	52
8	Chinstrap	Torgersen	0
9	Gentoo	Torgersen	0

Now we can plot mydf.

```
ggplot(data=mydf, aes(x=island, y=Count, fill=species)) +
  geom_col(position = "dodge") +
  labs(title = "Side-by-side Barplot", x = "Island", y = "Count")
```

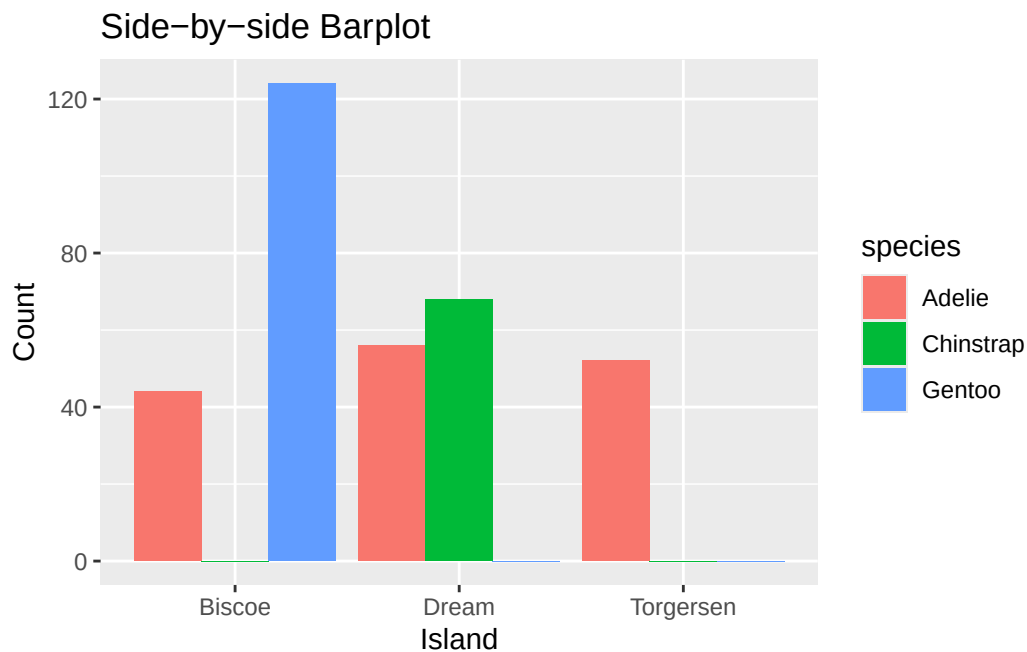


Figure 3.40: Plotting a contingency table using a side-by-side bar plot.

3.3.6 Two Numeric Variables

Use a scatter plot to visualize the relationship between two numeric variables. As said above, the revealed relationship can be linear and non-linear.

```
ggplot(iris, aes(x=Sepal.Width, y=Sepal.Length)) +  
  geom_point() +  
  labs(x="Sepal Width", y="Sepal Length", title = "A Scatter Plot") +  
  theme(plot.title = element_text(hjust = 0.5))
```

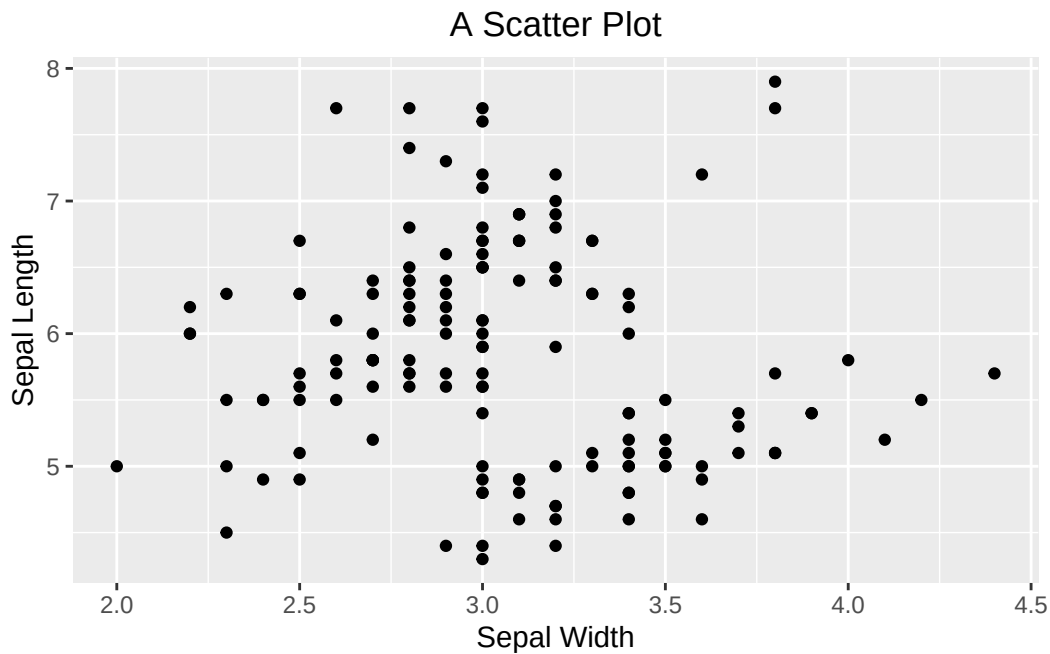


Figure 3.41: Visualizing the relationship between two numeric variables using a scatter plot.

If you wanted to emphasize their linear relationship, it can be done convincingly by adding a regression line on top of the scatter plot. `ggplot2` spares you the effort to create a linear model like base R. Instead, it can be done by adding the `geom_smooth()`. You tell `geom_smooth(method = "lm", se = FALSE, color = "red")` to use a linear method (`lm`) for smoothing the data points. See the code below:

```
ggplot(iris, aes(x=Sepal.Width, y=Sepal.Length)) +  
  geom_point() +  
  geom_smooth(method = "lm", se = FALSE, color = "red") +
```

```
labs(x="Sepal Width", y="Sepal Length", title = "Scatter Plot with the Regression Line") +
theme(plot.title = element_text(hjust = 0.5))
```

``geom_smooth()`` using formula = 'y ~ x'

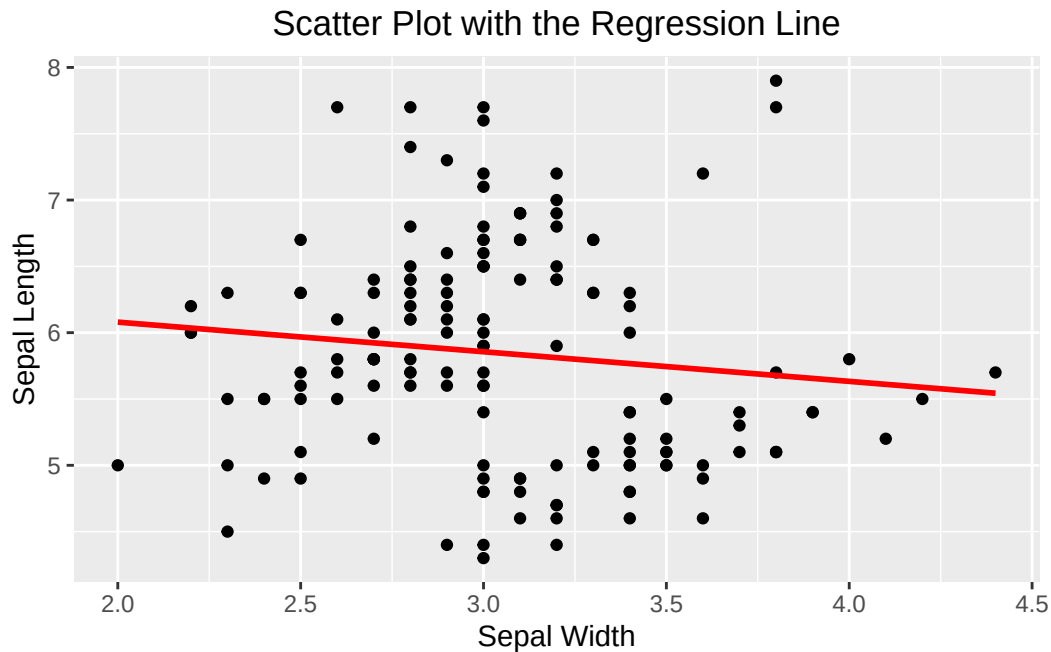


Figure 3.42: A scatter plot overlaid with a regression line.

If you see this message “`geom_smooth()` using formula = ‘y ~ x’”, it is harmless, and informational. `ggplot2` is just to let you know what the response and predictor are used by `geom_smooth()` in creating the regression line.

3.3.7 Multipanel Scatter Plot

Often times before multiple linear regression, it is recommended to visualize the relationship between the response variable and the independent variables pairwise, or the linear relationship between independent variables. The latter is important, as it may cause the **collinearity** issue if independent variables are linearly correlated.

A multipanel scatter plot can do this. However, `ggplot2` does not have a geometry function for it. The simplest way is to use the `ggpairs()` function provided by the `GGAly` package, which requires installation, if you have not done so.

Suppose, you are interested in the pairwise relationship between all numeric variables in `iris`. It can easily be done using the `ggpairs()` plot function. Color is the attribute used to distinguish species. `progress = FALSE` is to suppress the printing of the progress bar.

```
library(GGally)
ggpairs(iris, aes(color = Species), title = "A Multi-panel Scatter Plot", progress = FALSE)
```

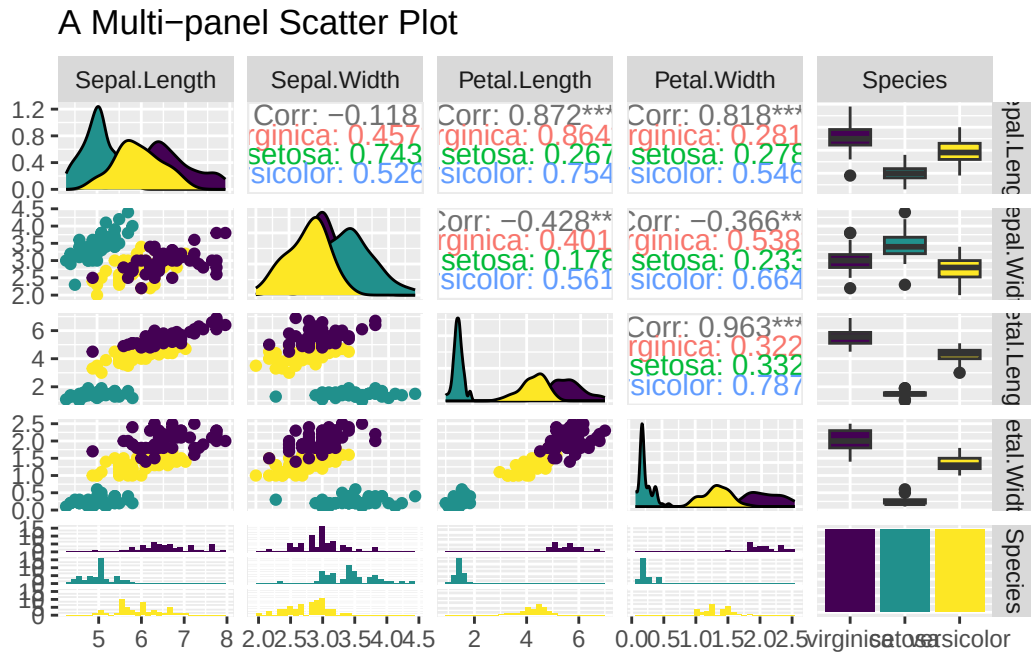


Figure 3.43: A Multipanel Scatter Plot

3.4 Summary

This is big chapter that has discussed an array of plot functions both from base R and `ggplot2`. It is more like a cookbook of various visualization recipes. So, instead of listing all the functions here, it is helpful to list several useful resources that you can fine-tune the graphs yourselves.

pch

If you type `help(pch)` at the console, R shows the `pch` section of the help page for `points`. Below is a snapshot of `pch` values and the corresponding plotting characters:

Color

R supports a large number of colors and palettes. [R Graph Gallery](#) is a great resource to see what choices are available and their names .

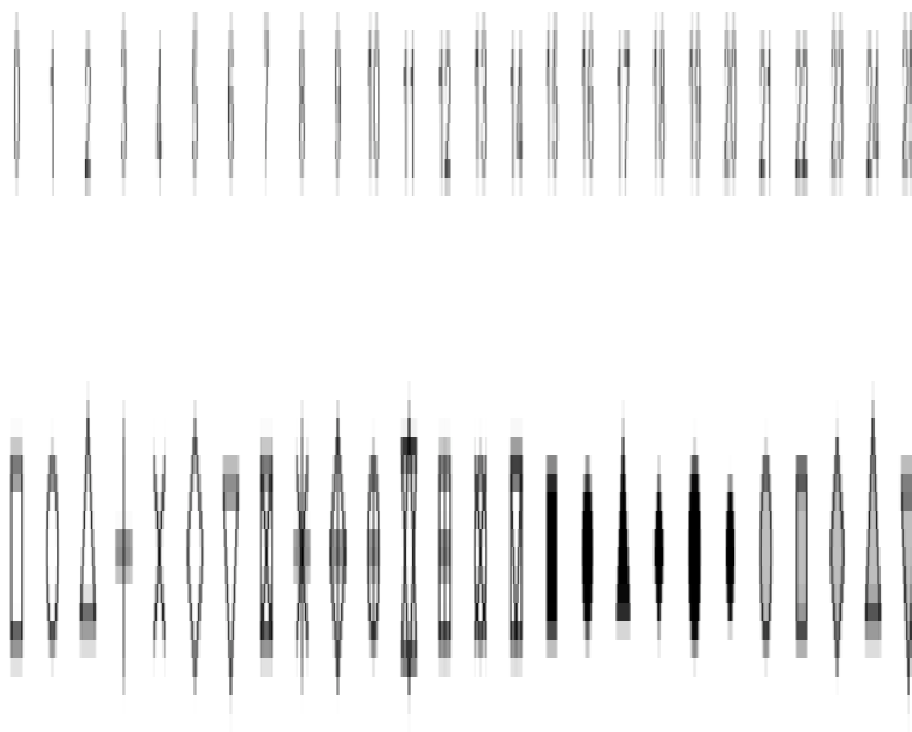


Figure 3.44: pch values

Part II

Part II

4 Dive Deeper

So far, we have discussed the basic functions of R that are sufficient for most situations. Here, we will discuss several intermediate level features that are practical, and yet utilize additional R functions. That includes the how to create summary tables from published work, converting a dataset from a wide format to a long format, and vice versa, the family of `apply()` functions, and a second look at `group_by()`.

Learning Objectives:

- Recreate a summary table from published work in R.
- Create a small data frame with data in R.
- Load datasets into R.
- Convert a dataset from a wide format to a long format.
- Convert a dataset from a long format to a wide format.
- Utilize the family of `apply()` functions.
- Revisit `group_by()`.

4.1 Recreate a Summary Table in R

If you see a data summary table as one in Figure ([@ref\(fig:Fig4-1\)](#)), and you may want to reproduce the analysis. Unless the raw data, i.e., the data for each sample, is available, it cannot be reproduced in R using the `table()` or `group_by()` functions. In such a scenario, you will learn the way to create an R table in this section.

```
`summarise()` has regrouped the output.  
i Summaries were computed grouped by agegp and tobgp.  
i Output is grouped by agegp.  
i Use `summarise(.groups = "drop_last")` to silence this message.  
i Use `summarise(.by = c(agegp, tobgp))` for per-operation grouping  
  (`?dplyr::dplyr_by`) instead.
```

```
# A tibble: 4 x 4
# Groups:   agegp [1]
  agegp tobgp Controls Cases
<ord> <ord>      <dbl> <dbl>
1 45-54 0-9g/day      90     14
2 45-54 10-19        44     13
3 45-54 20-29        25      8
4 45-54 30+          8     11
```

A

Heart Disease	Smoking Status			
	Formerly	Never	Smoke	Unknown
0	808	1802	728	1496
1	77	90	61	48

B

Group	Daily Tobacco Consumption	Controls	Cases
A	0-9g/day	90	14
B	10-19	44	13
C	20-29	25	8
D	30+	8	11

Figure 4.1: Recreate summary tables.

Here you will create the two tables in R as shown in Figure (@ref(fig:Fig4-1)). Let us begin with the first table, which consists of four columns and two rows. As all the values are numeric, you can create a numeric vector for each row using the combine or collate function `c()` (@ref(vector)). But in here, you will specify the vectors a bit differently. To include the column names in the final table, you have to associate a name to each value for the first row. For the subsequent rows, make sure the order of the values corresponds to the intended columns. As a result, each column in the final table will have a name when rows are combined. Below is the R code to create two rows with names using a vector:

```
row1 <- c(heart_disease=0, formerly=808, never=1802, smoke=728, unknown=1496)
row2 <- c(1, 77, 90, 61, 48)
```

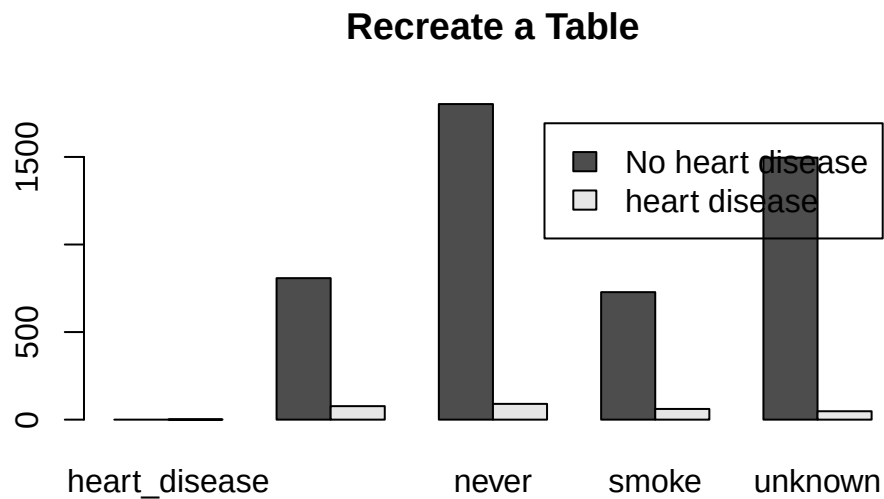
Next, you create a table by stacking up the rows using the `rbind()` function as shown below.

```
my_tab1 <- rbind(row1, row2)
my_tab1
```

```
      heart_disease formerly never smoke unknown
row1              0      808  1802   728   1496
row2              1       77    90    61     48
```

For example, you can plot the data, perform a χ^2 test of independence, etc.

```
barplot(my_tab1, main="Recreate a Table", beside=T, legend.text = c("No heart disease", "heart disease"))
```



```
chisq.test(my_tab1)
```

Warning in `chisq.test(my_tab1)`: Chi-squared approximation may be incorrect

Pearson's Chi-squared test

```
data: my_tab1  
X-squared = 61.967, df = 4, p-value = 1.119e-12
```

For the second table, each row contains a mixture of character and numeric values. Therefore, vector objects cannot be used to represent rows `@ref(atomic-and-object-variables)`. Instead, use `list` objects for the **heterogeneous** rows. Below is an example to define rows as list objects:

```
A <- list(group="A", daily_toba="0-9", ncontrols=90, ncases=14)  
B <- list("B", "10-19", 44, 13)  
C <- list("C", "20-29", 25, 8)  
D <- list("D", "30+", 8, 11)
```

Similarly, rows as lists are stacked up together to form a table.

```
my_tab2 <- rbind(A,B,C,D)  
my_tab2
```

	group	daily_toba	ncontrols	ncases
A	"A"	"0-9"	90	14
B	"B"	"10-19"	44	13
C	"C"	"20-29"	25	8
D	"D"	"30+"	8	11

4.2 Create a small data frame with data in R

In this section, you will learn two approaches to store a small dataset in an R's `data.frame` object. If you have a large dataset, this approach might not be the best choice. Instead, you should prepare the dataset using a spreadsheet software, e.g., Google Sheets or Microsoft Excel. Read the next section for more details.

4.2.1 Use `data.frame()` Function

Besides reproducing a table, you might want to create a small dataset as a data frame in R. It can be done in two steps, like creating a table above. The first step is to define each column (variable) as a vector, and then use the `data.frame()` function to combine them into a data

frame. Here is an example to create a data frame that consists of two columns (group, and systolic blood pressure (sbp)), where each column contains eight samples. The first step is to create two vectors, one for each column

```
group_data <- c("A","A","A","A","A","B","B","B")
sbp_data <- c(120,135,133,128,130,150,160,158)
```

Next, use the `data.frame()` function to combine the vectors into a data frame.

```
small_df1 <- data.frame(group = group_data,
                        sbp = sbp_data)
small_df1
```

	group	sbp
1	A	120
2	A	135
3	A	133
4	A	128
5	A	130
6	B	150
7	B	160
8	B	158

4.2.2 Use `read.table()` Function

An equivalent approach is to type out the values in columns as the example below, and feed it to the `read.table()` function through the `text=` parameter.

```
content = ("
group sbp
A 120
A 135
A 133
A 128
A 130
B 150
B 160
B 158
")

small_df2 <- read.table(text = content, header = TRUE)
small_df2
```

	group	sbp
1	A	120
2	A	135
3	A	133
4	A	128
5	A	130
6	B	150
7	B	160
8	B	158

4.3 Load datasets into R

In this section, you will learn several ways to load a data file into R. Data files are usually in two formats: comma-separated (.csv) or tab-separated (.txt or .tsv). In a .csv file, comma (,) is used as the separator to separate data in one column from the other column as shown in Figure 4-2A (@ref(fig:Fig4-2)). The other commonly used separator is the tab symbol, which is usually invisible, but it is made visible deliberately in Figure 4-2B.

A

Input File

```
id,gender,age,hypertension,heart_disease,ever_married,work_ty
9046,Male,67,0,1,Yes,Private,Urban,228.69,36.6,formerly smoke
51676,Female,61,0,0,Yes,Self-employed,Rural,202.21,N/A,never
31112,Male,80,0,1,Yes,Private,Rural,105.92,32.5,never smoked,
60182,Female,49,0,0,Yes,Private,Urban,171.23,34.4,smokes,1
1665,Female,79,1,0,Yes,Self-employed,Rural,174.12,24,never sm
56669,Male,81,0,0,Yes,Private,Urban,186.21,29,formerly smoked
53882,Male,74,1,1,Yes,Private,Rural,70.09,27.4,never smoked,1
10434,Female,69,0,0,No,Private,Urban,94.39,22.8,never smoked,
27419,Female,59,0,0,Yes,Private,Rural,76.15,N/A,Unknown,1
60491,Female,78,0,0,Yes,Private,Urban,58.57,24.2,Unknown,1
12109,Female,81,1,0,Yes,Private,Rural,80.43,29.7,never smoked
12095,Female,61,0,1,Yes,Govt_job,Rural,120.46,36.8,smokes,1
12175,Female,54,0,0,Yes,Private,Urban,104.51,27.3,smokes,1
```

B

Input File

year	↵	boys	↵	girls
1629	↵	5218	↵	4683
1630	↵	4858	↵	4457
1631	↵	4422	↵	4102
1632	↵	4994	↵	4590
1633	↵	5158	↵	4839
1634	↵	5035	↵	4820
1635	↵	5106	↵	4928
1636	↵	4917	↵	4605
1637	↵	4703	↵	4457
1638	↵	5359	↵	4952
1639	↵	5366	↵	4784
1640	↵	5518	↵	5332
1641	↵	5470	↵	5200

Figure 4.2: File formats.

4.3.1 `read.csv()` Function

4.3.2 Import data in RStudio

4.4 `group_by()`

4.5 Different apply Functions

4.6 Wide to Long

Use AirPassengers dataset

Bibliography